



VHDL入门教程

目录

第0部分：VHDL语言基础教程

- A. VHDL语言基础
 - A.1 VHDL文字规则
 - A.2 数据对象
 - A.3 数据类型
 - A.4 运算符
 - A.5 属性
- B. 基本描述语句
 - B.1 顺序语句
 - B.2 并行语句

第一部分：组合逻辑器件

1. 与门 AND
2. 或门 OR
3. 非门 NOT
4. 与非门 NAND
5. 或非门 NOR
6. 异或门 XOR
7. 同或门 XNOR
8. 三态门
9. 半加器
10. 全加器
11. 4位串行进位加法器
12. 超前进位加法器 CLA
13. BCD加法器
14. 串行加法器
15. 半减器
16. 全减器

17. BCD减法器
18. 1位比较器
19. 4位比较器
20. 8位比较器
21. 2选1MUX
22. 4选1MUX
23. 8选1MUX
24. 16选1MUX
25. 1-2 DEMUX
26. 1-4 DEMUX
27. 1-8 DEMUX
28. 2-4译码器
29. 3-8译码器
30. 4-16译码器
31. 74138风格译码器
32. 普通编码器
33. 优先编码器
34. 74148优先编码器
35. 奇校验器
36. 偶校验器
37. 校验发生器
38. 校验检测器
39. 二进制转格雷码
40. 格雷码转二进制
41. BCD转二进制
42. 二进制转BCD
43. 七段译码器
44. 七段动态扫描驱动
45. 简单ALU
46. 74181风格ALU

第二部分：时序逻辑器件

47. SR锁存器
48. D锁存器
49. D触发器
50. JK触发器

- 51. [T触发器](#)
- 52. [RS触发器](#)
- 53. [4位寄存器](#)
- 54. [8位寄存器](#)
- 55. [通用寄存器](#)
- 56. [左移寄存器](#)
- 57. [右移寄存器](#)
- 58. [双向移位寄存器](#)
- 59. [通用移位寄存器](#)
- 60. [串行输入并行输出](#)
- 61. [并行输入串行输出](#)
- 62. [异步计数器](#)
- 63. [同步计数器](#)
- 64. [十进制计数器](#)
- 65. [任意进制计数器](#)
- 66. [可逆计数器](#)
- 67. [环形计数器](#)
- 68. [Johnson计数器](#)
- 69. [二分频器](#)
- 70. [四分频器](#)
- 71. [任意分频器](#)
- 72. [Moore状态机](#)
- 73. [Mealy状态机](#)
- 74. [序列检测器1011](#)
- 75. [序列检测器1101](#)
- 76. [ROM](#)
- 77. [单端口RAM](#)
- 78. [双端口RAM](#)
- 79. [FIFO](#)

第〇部分：VHDL语言基础教程

A. VHDL语言基础

A.1 VHDL文字规则

VHDL (VHSIC Hardware Description Language) 是一种硬件描述语言，对大小写不敏感（`signal` 与 `SIGNAL` 等价）。其基本文字规则如下：

标识符

- **基本标识符**：以字母开头，后跟字母、数字或下划线 `_`，不允许连续两个下划线，不允许下划线结尾。例：`data_in`、`clk_100M`、`Q0`
- **扩展标识符**：以反斜杠 `\` 包围，内部可含任意字符。例：`\-WE\`、`\clk!\`

注释

VHDL 注释以 `--` 开头，持续到行尾。

```
-- 这是一行注释
Y <= A and B;    -- 输入A与B相与，赋值给Y
```

数字表示

格式	示例	含义
十进制整数	42	十进制42
基表示法	2#1010#	二进制1010 = 十进制10
基表示法	16#FF#	十六进制FF = 十进制255
实数	3.14、1.0e-9	浮点数（仿真用）
字符	'A'、'1'	单个字符
字符串	"1010"	多位字符串
位字符串	B"1010"	二进制

格式	示例	含义
位字符串	X"FF"	十六进制
位字符串	O"77"	八进制

关键字

VHDL有保留关键字，不可用作标识符。常用关键字包括：

```
entity architecture signal variable constant type subtype begin end process if
else elsif case when loop for while next exit wait null not and or nand nor
xor xnor in out inout buffer port generic map component block generate library
use all others range downto to rising_edge falling_edge after transport
inertial report assert severity
```

A.2 数据对象

VHDL中有四类数据对象：**信号 (Signal)**、**变量 (Variable)**、**常量 (Constant)** 和**文件 (File)**。

信号 Signal

信号是VHDL中最核心的数据对象，用于实体间的通信和时序逻辑。信号赋值使用 `<=`。

```
-- 信号声明
signal data_bus : STD_LOGIC_VECTOR(7 downto 0);
signal clk_in   : STD_LOGIC;
signal counter  : INTEGER range 0 to 255;
signal ready    : STD_LOGIC := '0';    -- 赋初值（仿真用）
```

信号特点：

- 信号赋值不会立即生效，而是在进程执行完毕（或遇到 `wait`）后才更新
- 信号具有延时特性（delta delay），在同一个delta cycle内可能有多个信号更新

- 信号在architecture内声明，在package内可声明全局信号

变量 Variable

变量仅在进程（process）或子程序（function/procedure）内部使用。变量赋值使用 `:=`。

```
process(clk)
    variable temp_sum : STD_LOGIC_VECTOR(7 downto 0);
    variable count    : INTEGER := 0;
begin
    temp_sum := A + B;      -- 变量赋值立即生效
    count   := count + 1;
end process;
```

变量特点：

- 变量赋值立即生效，无延时
- 变量只在定义它的进程/子程序内可见
- 综合工具通常将其映射为组合逻辑寄存器或临时连线

常量 Constant

常量在声明时赋值，运行期间不可更改。

```
constant MAX_COUNT : INTEGER := 1024;
constant PI_VALUE  : REAL    := 3.14159;
constant VCC       : STD_LOGIC := '1';
constant GND       : STD_LOGIC := '0';
constant IDLE      : STD_LOGIC_VECTOR(1 downto 0) := "00";
```

信号与变量的关键区别

特性	Signal	Variable
赋值符号	<code><=</code>	<code>:=</code>
生效时机	延迟生效（进程结束/ Δ 延时后）	立即生效

特性	Signal	Variable
作用范围	architecture/package内	当前process/子程序内
综合结果	有记忆（可生成触发器/连线）	一般为组合逻辑连线
声明位置	architecture声明区	process声明区

关键陷阱：在同一个process中，对signal的多次赋值只有最后一次生效（因为信号赋值延迟生效，后续赋值覆盖前者）。而variable每次赋值立即生效，后续语句立刻看到新值。

```
-- ✗ 错误理解：期望Q依次变为A, B, C
process(clk)
begin
    if rising_edge(clk) then
        Q <= A;    -- 被下一句覆盖！
        Q <= B;    -- 被下一句覆盖！
        Q <= C;    -- 最终Q = C
    end if;
end process;

-- ✔ 变量立即生效
process(clk)
    variable temp : STD_LOGIC;
begin
    if rising_edge(clk) then
        temp := A;    -- temp = A
        temp := B;    -- temp = B（覆盖A）
        temp := C;    -- temp = C（覆盖B）
        Q <= temp;    -- Q最终 = C
    end if;
end process;
```



signal与variable的核心区别

	Signal	Variable
声明位置	architecture	process内部
作用域	整个architecture	声明它的process/子程序内
赋值符号	<code><=</code>	<code>:=</code>
生效时间	延迟（process执行完或wait）	立即生效
综合结果	可能生成触发器或连线	通常为连线
多次赋值	最后一句生效	每次赋值都生效

关键测试代码：

```
-- 比较signal和variable的区别
process(clk)
    variable var : STD_LOGIC;
    signal sig : STD_LOGIC;
begin
    if rising_edge(clk) then
        var := '0';           -- var立即=0
        var := '1';           -- var立即=1（覆盖）
        Q1 <= var;             -- Q1 = '1'（最后一次赋值）

        sig <= '0';           -- 延迟生效
        sig <= '1';           -- 延迟生效（覆盖前句）
        Q2 <= sig;             -- Q2 = sig旧值（不是'1'！）
    end if;
end process;
-- 结果：Q1 = '1'（变量），Q2 = '0'（信号旧值，上周期sig的值）
```


初始值与复位

```
-- 信号声明时赋初值：仅对仿真有效，不可被综合工具映射为初始状态
signal Q : STD_LOGIC := '0';    -- 仿真初值，综合后寄存器初始值不确定

-- 正确做法：使用复位信号
process(clk, rst)
begin
    if (rst = '1') then
        Q <= '0';                -- 复位值，综合为异步清零
    elsif rising_edge(clk) then
        Q <= D;
    end if;
end process;
```

A.3 数据类型

VHDL是强类型语言，不同数据类型之间不可直接运算（需类型转换）。

标准类型（包IEEE.STD_LOGIC_1164）

类型	说明	示例
STD_LOGIC	9值逻辑	'0', '1', 'Z', 'X', 'U', 'W', 'L', 'H', '-'
STD_LOGIC_VECTOR	STD_LOGIC数组	"1010", x"FF"
STD_ULOGIC	9值逻辑（无驱动竞争检查）	'0', '1', 'Z', 'X', ...
STD_ULOGIC_VECTOR	STD_ULOGIC数组	"1010"

STD_LOGIC九值含义：

值	含义	说明
'U'	Uninitialized	未初始化
'X'	Forcing Unknown	强未知（多驱动冲突）

值	含义	说明
'0'	Forcing 0	强逻辑0
'1'	Forcing 1	强逻辑1
'Z'	High Impedance	高阻态（三态总线）
'W'	Weak Unknown	弱未知
'L'	Weak 0	弱逻辑0
'H'	Weak 1	弱逻辑1
'-'	Don't Care	无关项（综合可优化）

基本类型（包STANDARD）

类型	范围	示例
BOOLEAN	TRUE , FALSE	TRUE
BIT	'0' , '1'	'1'
BIT_VECTOR	BIT数组	"1010"
INTEGER	-2147483647 ~ +2147483647	42 , -3
NATURAL	0 ~ +2147483647	0 , 255
POSITIVE	1 ~ +2147483647	1 , 1024
REAL	浮点数（不可综合）	3.14
CHARACTER	单个字符	'A'
STRING	字符数组	"hello"
TIME	时间（不可综合）	10 ns , 1 us
SEVERITY_LEVEL	NOTE/WARNING/ERROR/FAILURE	ERROR

常用包类型

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;           -- STD_LOGIC
use IEEE.NUMERIC_STD.ALL;              -- unsigned, signed
use IEEE.NUMERIC_BIT.ALL;              -- unsigned(bit), signed(bit)
use IEEE.STD_LOGIC_UNSIGNED.ALL;       -- 无符号运算（不推荐新设计）
use IEEE.STD_LOGIC_SIGNED.ALL;         -- 有符号运算（不推荐新设计）
use IEEE.MATH_REAL.ALL;                -- 数学函数（不可综合）
```

NUMERIC_STD包（推荐用于新设计）

```
library IEEE;
use IEEE.NUMERIC_STD.ALL;

signal a_unsigned : unsigned(7 downto 0);    -- 无符号8位
signal b_signed   : signed(7 downto 0);       -- 有符号8位
signal c_std      : STD_LOGIC_VECTOR(7 downto 0);

-- 类型转换
a_unsigned <= unsigned(c_std);                -- std_logic_vector → unsigned
c_std      <= std_logic_vector(a_unsigned);   -- unsigned → std_logic_vector
b_signed   <= signed(c_std);                  -- std_logic_vector → signed

-- 运算
a_unsigned <= a_unsigned + 1;
b_signed   <= b_signed - 1;
```

用户自定义类型

```
-- 枚举类型
type state_type is (IDLE, READ, WRITE, DONE);
signal state : state_type;

-- 数组类型
type byte_array is array (0 to 15) of STD_LOGIC_VECTOR(7 downto 0);
signal ram : byte_array;

-- 二维数组
type matrix_8x8 is array (0 to 7, 0 to 7) of STD_LOGIC;

-- 记录类型
type packet is record
    addr : STD_LOGIC_VECTOR(7 downto 0);
    data : STD_LOGIC_VECTOR(15 downto 0);
    valid : STD_LOGIC;
end record;
signal current_packet : packet;

-- 子类型（限制范围）
subtype digit is INTEGER range 0 to 9;
subtype byte is STD_LOGIC_VECTOR(7 downto 0);
```

A.4 运算符

VHDL运算符按优先级从高到低排列：

优先级	类别	运算符	说明
最高	杂项	<code>**</code> <code>abs</code> <code>not</code>	幂运算、绝对值、逻辑非
高	乘法类	<code>*</code> <code>/</code> <code>mod</code> <code>rem</code>	乘、除、取模、取余
中	符号	<code>+</code> <code>-</code>	正、负

优先级	类别	运算符	说明
中	加法类	<code>+</code> <code>-</code> <code>&</code>	加、减、拼接
低	关系类	<code>=</code> <code>/=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	等于、不等、小于、小于等于、大于、大于等于
最低	逻辑类	<code>and</code> <code>or</code> <code>nand</code> <code>nor</code> <code>xor</code> <code>xnor</code>	与、或、与非、或非、异或、同或
—	移位	<code>sll</code> <code>srl</code> <code>sla</code> <code>sra</code> <code>rol</code> <code>ror</code>	逻辑左/右移、算术左/右移、循环左/右移

拼接运算符 `&`

```
signal a : STD_LOGIC_VECTOR(3 downto 0);
signal b : STD_LOGIC_VECTOR(3 downto 0);
signal c : STD_LOGIC_VECTOR(7 downto 0);

c <= a & b;           -- "aaaa" & "bbbb" = "aaaabbbb"
c <= "1010" & "0101"; -- = "10100101"
c <= '1' & a & "000"; -- = '1' & a(3) & a(2) & a(1) & a(0) & '0' & '0' & '0'
```

注意要点

- 1. `**` 运算在部分综合工具中受限，建议移位的写法
- 2. `/` `mod` `rem` 除数必须为2的幂，否则综合困难
- 3. 关系类运算中 `<` 和 `<=` 需根据上下文区分（信号赋值用 `<=`，比较用 `<=` 在括号内）
- 4. 同优先级的运算符从左到右结合
- 5. 使用括号可以改变优先级

A.5 属性

VHDL提供丰富的属性（Attribute）用于获取类型、数组、信号的相关信息。

数组属性

```
signal data : STD_LOGIC_VECTOR(7 downto 0);
```

```
data'LEFT      -- = 7   (左边界)
data'RIGHT     -- = 0   (右边界)
data'HIGH      -- = 7   (上边界)
data'LOW       -- = 0   (下边界)
data'RANGE     -- = 7 downto 0
data'REVERSE_RANGE -- = 0 to 7
data'LENGTH    -- = 8   (长度)
data'ASCENDING -- = FALSE (是否升序)
```

信号属性（事件检测）

```
signal clk : STD_LOGIC;
```

```
clk'EVENT      -- clk值发生变化时为TRUE
clk'LAST_VALUE -- clk上一次的值
clk'STABLE(5 ns) -- clk在过去5ns内未变化为TRUE
clk'DELAYED(10 ns) -- clk延时10ns后的版本

-- 常用上升沿/下降沿检测
rising_edge(clk) -- 等价于 clk'EVENT and clk = '1'
falling_edge(clk) -- 等价于 clk'EVENT and clk = '0'

-- 原始写法（不推荐，建议用rising_edge/falling_edge函数）
if (clk'event and clk = '1') then -- 上升沿
if (clk'event and clk = '0') then -- 下降沿
```

类型属性

```
type state_type is (IDLE, READ, WRITE, DONE);

state_type'POS(READ)      -- = 1  (READ在枚举中的位置, 0起始)
state_type'VAL(2)         -- = WRITE (位置2对应的值)
state_type'LEFT           -- = IDLE
state_type'RIGHT          -- = DONE
state_type'LENGTH         -- = 4
state_type'SUCC(READ)     -- = WRITE (后继)
state_type'PRED(WRITE)    -- = READ  (前驱)
```

B. 基本描述语句

VHDL语句分两大类：**顺序语句 (Sequential)** 和 **并行语句 (Concurrent)**。

- **顺序语句**：出现在process、function、procedure内部，从上到下依次执行
- **并行语句**：出现在architecture的begin...end之间，所有语句同时执行

B.1 顺序语句

顺序语句必须在process、function或procedure内部使用。

B.1.1 顺序赋值语句

```
-- 变量赋值 (:=)，立即生效
variable temp : STD_LOGIC;
temp := '1';

-- 信号赋值 (<=)，延迟生效 (process执行完毕或wait后)
signal s : STD_LOGIC;
s <= '1';
```

重要：即使在process内部，对Signal赋值也是延迟生效的。同一process内对同一Signal多次赋值，只有最后一次有效。

B.1.2 IF语句

```
-- 单分支
if (condition) then
    -- 语句
end if;

-- 二分支
if (condition) then
    -- 语句1
else
    -- 语句2
end if;

-- 多分支
if (condition1) then
    -- 语句1
elsif (condition2) then
    -- 语句2
elsif (condition3) then
    -- 语句3
else
    -- 默认语句
end if;
```

示例：


```
-- D触发器
process(clk, rst)
begin
    if (rst = '1') then
        Q <= '0';
    elsif rising_edge(clk) then
        if (ce = '1') then
            Q <= D;
        end if;
    end if;
end process;
```

注意：

- `if` 相互嵌套会形成优先级链，综合后产生级联MUX，对时序不利
- 若不需要优先级，优先使用 `case` 语句
- 组合逻辑的 `if` 必须有 `else`（或为所有信号赋默认值），否则综合出**锁存器** (Latch)

```
-- ✗ 组合逻辑缺else → 综合出Latch
process(A, B, sel)
begin
    if (sel = '1') then
        Y <= A;
    end if;
    -- 缺else → Y保持原值 → Latch!
end process;
```

```
-- ✓ 加else避免Latch
process(A, B, sel)
begin
    if (sel = '1') then
        Y <= A;
    else
        Y <= B;
    end if;
end process;
```

```
-- ✓ 用默认赋值
process(A, B, sel)
begin
    Y <= B;
    -- 默认赋值
    if (sel = '1') then
        Y <= A;
    end if;
end process;
```

B.1.3 CASE语句

```
case expression is
  when value1 =>
    -- 语句1
  when value2 =>
    -- 语句2
  when value3 | value4 =>      -- 多个值共享同一分支
    -- 语句3
  when value5 to value8 =>    -- 范围
    -- 语句4
  when others =>              -- 必须覆盖所有情况
    -- 默认语句
end case;
```

示例：

```
-- 七段译码器
process(bcd)
begin
  case bcd is
    when "0000" => seg <= "0111111"; -- 0
    when "0001" => seg <= "0000110"; -- 1
    when "0010" => seg <= "1011011"; -- 2
    when "0011" => seg <= "1001111"; -- 3
    when "0100" => seg <= "1100110"; -- 4
    when "0101" => seg <= "1101101"; -- 5
    when "0110" => seg <= "1111101"; -- 6
    when "0111" => seg <= "0000111"; -- 7
    when "1000" => seg <= "1111111"; -- 8
    when "1001" => seg <= "1101111"; -- 9
    when others => seg <= "0000000"; -- 必须！
  end case;
end process;
```

核心规则：

1. 必须覆盖所有可能情况： `when others` 不能省（除非所有枚举值都已覆盖）

2. 所有分支必须互斥
3. **每个分支必须为所有输出信号赋值**（或使用默认赋值），否则综合出Latch
4. `case` 无优先级，产生并行译码结构，通常优于 `if-elsif` 链

B.1.4 FOR-LOOP语句

`for-loop` 在综合时必须能确定循环次数（即循环边界在综合时为常数）。

```
-- 通用奇偶校验计算
process(data_in)
    variable parity : STD_LOGIC;
begin
    parity := '0';
    for i in data_in'RANGE loop
        parity := parity xor data_in(i);
    end loop;
    parity_out <= parity;
end process;
```

注意：

- 循环变量（如上面的 `i`）不需要声明，默认存在
- 循环边界必须是静态可确定的（如常量、generic参数、数组的range属性）
- `for-loop` 在综合时会被展开（unroll），产生独立硬件

```
-- 移位寄存器（for-generate不可用时的替代）
process(clk)
begin
    if rising_edge(clk) then
        reg(0) <= si;
        for i in 1 to 7 loop
            reg(i) <= reg(i-1);
        end loop;
    end if;
end process;
```

B.1.5 WHILE-LOOP语句

```
while (condition) loop
    -- 循环体
    if (condition2) then
        exit;    -- 跳出循环
    end if;
end loop;
```

注意： while-loop 在综合时条件必须能在编译时确定，否则综合困难。RTL综合中很少使用。

B.1.6 NEXT语句

跳过本次循环剩余部分，进入下一次迭代。

```
for i in 0 to 15 loop
    next when (i = 5);    -- 跳过i=5的情况
    data(i) <= data(i) + 1;
end loop;

for i in 0 to 15 loop
    next exit_loop when (i = 8);    -- 带标签的next
    -- ...
end loop;
```

B.1.7 EXIT语句

跳出当前循环。

```

for i in 0 to 15 loop
    if (found = '1') then
        exit;                -- 跳出整个循环
    end if;
end loop;

for i in 0 to 15 loop
    exit outer_loop when (done = '1');  -- 带标签的exit
end loop;

```

B.1.8 WAIT语句

暂停进程执行，直到指定条件满足。**不可综合（仅仿真用）**。

```

-- 三种形式
wait on sensitivity_list;      -- 等待敏感信号变化
wait until condition;         -- 等待条件为真
wait for time_expression;     -- 等待指定时间

-- 组合使用
wait on clk until clk = '1';   -- 等待clk上升沿
wait until rising_edge(clk);
wait for 10 ns;                -- 等待10纳秒
wait on A, B until (A = '1');  -- 等待A或B变化且A='1'

-- 含有wait的process不能有敏感信号列表
process
begin
    wait until rising_edge(clk);
    Q <= D;
    wait until rising_edge(clk);
    Q <= '0';
end process;

```

B.1.9 NULL语句

空操作，占位用。

```
case state is
  when IDLE =>
    if (start = '1') then
      state <= RUN;
    else
      null;                -- 什么都不做
    end if;
  when others =>
    null;
end case;
```

注意： `when others => null;` 用于case语句中不需要处理的剩余情况。此为顺序语句，综合后相当于“不驱动任何信号”。

Delta延时 (Δ Delay)

VHDL仿真采用**Delta延时**模型。在同一仿真时刻（如0ns），信号赋值在下一个Delta周期生效。这允许模拟零物理时间的“瞬时传播”，同时正确建模并发行为。

仿真时刻0ns：

Delta 0: clk跳变 → 触发process

Delta 1: process内信号赋值生效（`Q <= D`生效）

Delta 2: 组合逻辑使用新Q值计算，可能触发更多process

...

直到无更多事件 → 进入下一物理时刻（下一个时钟周期）

实际影响：

- 综合工具忽略delta延时（综合后无延时概念）
- 仿真中 `after` 语句指定物理延时，不指定则为delta延时
- 无限delta循环会导致仿真挂死

传输延时 vs 惯性延时

```
-- 传输延时 (Transport)：所有脉冲都能通过
Y <= transport A after 5 ns;

-- 惯性延时 (Inertial, 默认)：窄脉冲被滤除
Y <= A after 5 ns;           -- 等效于 inertial
Y <= reject 3 ns inertial A after 5 ns; -- 脉冲<3ns被滤除
```

综合注意事项： `after` 和 `transport` 不可综合，仅用于仿真。综合工具忽略所有延时语句。

避免意外Latch的检查清单

在组合逻辑process中，以下情况会综合出Latch：

1. **if没有else** → 当条件不满足时输出保持不变 → Latch
2. **case没覆盖所有情况且无others** → 未覆盖情况输出保持 → Latch
3. **case分支内没有为所有输出赋值** → 某输出在该分支"遗漏"赋值 → Latch

解决方案：


```

-- 方案一：完整赋值
process(A, sel)
begin
    if (sel = '1') then
        Y <= A;
    else
        Y <= '0';          -- 所有路径都赋值
    end if;
end process;

-- 方案二：默认赋值（推荐）
process(A, sel)
begin
    Y <= '0';              -- 默认值
    if (sel = '1') then
        Y <= A;            -- 特定条件下覆盖
    end if;
end process;

-- 方案三：case必须含others
case sel is
    when "00" => Y <= D0;
    when "01" => Y <= D1;
    when "10" => Y <= D2;
    when "11" => Y <= D3;
    when others => Y <= '0'; -- 必须！
end case;

```

B.2 并行语句

并行语句出现在architecture的 `begin...end` 之间，所有语句**同时（并发）**执行，与书写顺序无关。

B.2.1 并行信号赋值语句

```
-- 简单并行赋值
Y <= A and B;

-- 条件信号赋值 (when-else)
Y <= D0 when S = "00" else
    D1 when S = "01" else
    D2 when S = "10" else
    D3;

-- 选择信号赋值 (with-select)
with S select
    Y <= D0 when "00",
        D1 when "01",
        D2 when "10",
        D3 when "11",
        '0' when others;
```

when-else vs with-select 区别：

特点	when-else	with-select
优先级	有优先级（先判断的在前）	无优先级（并行）
完整性	不需要覆盖所有情况	必须覆盖所有情况（含others）
综合结果	级联MUX	并行译码
等效性	等效于if-elsif链	等效于case

```

-- when-else: 有优先级
Y <= "111" when (priority(2) = '1') else
    "110" when (priority(1) = '1') else
    "101" when (priority(0) = '1') else
    "000";

-- with-select: 必须完整
with func select
    Y <= A + B when "00",
        A - B when "01",
        A and B when "10",
        not A when "11",
        (others => '0') when others;    -- 必须!

```

B.2.2 PROCESS语句

process是最核心的并行语句，一个architecture内可以有多个process，它们之间并发执行。

结构：

```

label: process(sensitivity_list)    -- label可选，敏感列表可选
    -- 声明区：变量、常量
    variable temp : STD_LOGIC;
begin
    -- 顺序语句区
    -- ...
end process;

```

敏感列表规则：

- 组合逻辑process：敏感列表必须包含所有被读取的信号，否则综合结果与仿真行为不一致（仿真时可能漏触发）
- 时序逻辑process：敏感列表通常只有时钟和异步复位
- 如果process内有 wait 语句，则**不能**有敏感列表

-- 组合逻辑：所有输入信号均应入敏感列表

```
process(A, B, sel)
begin
    if (sel = '1') then
        Y <= A;
    else
        Y <= B;
    end if;
end process;
```

-- 时序逻辑：仅clk和异步rst

```
process(clk, rst_n)
begin
    if (rst_n = '0') then
        Q <= '0';
    elsif rising_edge(clk) then
        Q <= D;
    end if;
end process;
```

-- 带wait（不可有敏感列表）

```
process
begin
    wait until rising_edge(clk);
    Q <= D;
end process;
```

B.2.3 元件例化语句

元件例化（Component Instantiation）用于结构化设计，将已定义的entity作为子模块例化到当前设计中。

传统方式：

```
architecture Structural of TOP is
```

```
-- 第一步：元件声明（或在package中声明）
```

```
component AND2 is
```

```
    Port (
```

```
        A : in  STD_LOGIC;
```

```
        B : in  STD_LOGIC;
```

```
        Y : out STD_LOGIC
```

```
    );
```

```
end component;
```

```
signal wire_and1 : STD_LOGIC;
```

```
signal wire_and2 : STD_LOGIC;
```

```
begin
```

```
-- 第二步：元件例化
```

```
U1: AND2
```

```
    port map (
```

```
        A => input_a,
```

```
        B => input_b,
```

```
        Y => wire_and1
```

```
    );
```

```
U2: AND2
```

```
    port map (
```

```
        A => input_c,
```

```
        B => input_d,
```

```
        Y => wire_and2
```

```
    );
```

```
end Structural;
```

直接例化（推荐，VHDL-93起）： 无需component声明。

```

architecture Structural of TOP is
    signal wire_and1 : STD_LOGIC;
    signal wire_and2 : STD_LOGIC;
begin
    U1: entity work.AND2
        port map (
            A => input_a,
            B => input_b,
            Y => wire_and1
        );

    U2: entity work.AND2
        port map (
            A => input_c,
            B => input_d,
            Y => wire_and2
        );
end Structural;

```

端口映射的两种方式：

```

-- 1. 命名关联 (Named Association)：推荐，不易出错
U1: AND2 port map (A => sig_a, B => sig_b, Y => sig_y);

-- 2. 位置关联 (Positional Association)：按Port声明顺序
U1: AND2 port map (sig_a, sig_b, sig_y); -- 必须顺序一致

-- 3. 混合方式
U1: AND2 port map (sig_a, sig_b, Y => sig_y);

-- 4. 未连接端口
U1: AND2 port map (A => sig_a, B => sig_b, Y => open); -- open表示不连接

```

B.2.4 块语句 BLOCK

Block语句用于将architecture内的并行语句分组。

```

architecture Behavioral of Example is
begin

    -- 块1: 数据输入处理
    input_block: block
        signal temp : STD_LOGIC_VECTOR(7 downto 0);
    begin
        temp <= A + B;
        Q1 <= temp;
    end block;

    -- 块2: 控制逻辑
    ctrl_block: block
    begin
        Q2 <= Q1 when en = '1' else (others => '0');
    end block;

end Behavioral;

```

Block vs Process: block内仍然是并行语句，仅仅是结构化分组。process内是顺序语句。

B.2.5 生成语句 GENERATE

Generate用于按规则重复生成并行结构。

for-generate (重复生成) :

```

-- 生成8个D触发器构成移位寄存器
GEN_SHIFT: for i in 0 to 6 generate
    FF: entity work.D_FLIPFLOP
        port map (
            CLK => clk,
            RST => rst,
            D   => reg(i+1),
            Q   => reg(i)
        );
end generate;

```

```

-- 生成N位加法器进位链
GEN_CARRY: for i in 0 to N-1 generate
    FA: entity work.FULL_ADDER
        port map (
            A   => A(i),
            B   => B(i),
            CIN => carry(i),
            SUM => SUM(i),
            COUT => carry(i+1)
        );
end generate;

```

if-generate（条件生成）：

```

GEN_ARRAY: for i in 0 to 7 generate

    -- 第一个元素特殊处理
    GEN_FIRST: if (i = 0) generate
        reg(i) <= si;
    end generate;

    -- 其他元素
    GEN_OTHER: if (i > 0) generate
        reg(i) <= reg(i-1);
    end generate;

end generate;

```


注意：

- `for-generate` 的循环变量不需声明，范围必须静态
-

组合逻辑 vs 时序逻辑的模板

组合逻辑模板：

```
process(所有输入信号)    -- 敏感列表必须完整
begin
    -- 为所有输出赋默认值（防止Latch）
    Y <= (others => '0');

    if (condition) then
        Y <= A;
    end if;
end process;
```

时序逻辑模板（异步复位）：

```
process(clk, rst_n)
begin
    if (rst_n = '0') then
        Q <= '0';
    elsif rising_edge(clk) then
        Q <= D;
    end if;
end process;
```

时序逻辑模板（同步复位）：

```

process(clk)
begin
    if rising_edge(clk) then
        if (rst_n = '0') then
            Q <= '0';
        else
            Q <= D;
        end if;
    end if;
end process;

```

通用设计注意事项

1. 包的使用顺序：

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;      -- 必须
use IEEE.NUMERIC_STD.ALL;         -- 推荐（替代STD_LOGIC_UNSIGNED）

```

****优先使用 NUMERIC_STD **而非 STD_LOGIC_UNSIGNED / STD_LOGIC_SIGNED**（后者非IEEE标准）。

2. clk'event vs rising_edge()：

```

-- 推荐写法
if rising_edge(clk) then    -- ✓ 正确处理X→1, Z→1等

-- 老旧写法
if (clk'event and clk = '1') then    -- ⚠ 有问题：X→1也触发，0→Z也触发

```

rising_edge() 专为RTL设计，正确处理从'0'/'L'到'1'/'H'的跳变。

3. 向量方向约定：

```

signal addr : STD_LOGIC_VECTOR(7 downto 0);    -- 7=MSB, 0=LSB（推荐）
signal addr : STD_LOGIC_VECTOR(0 to 7);        -- 0=MSB, 7=LSB

```

推荐统一使用 **downto** 约定，**7 downto 0** 中7是最高位。

4. 嵌套if的深度：过深的嵌套if会增加组合逻辑的级数，降低时序性能。优先考虑扁平化设计或用case替代。

5. 三态总线:

```
-- 内部三态（仅FPGA内部可用，不建议在新设计中使用）  
data <= data_a when en_a = '1' else (others => 'Z');  
data <= data_b when en_b = '1' else (others => 'Z');  
-- 多个三态驱动同一信号 → 仅在顶层IO使用
```

6. 异步复位 + 异步释放会带来毛刺风险，推荐用异步复位同步释放:

```
process(clk, rst_async)  
begin  
    if (rst_async = '1') then  
        rst_sync <= '1';          -- 同步器  
    elsif rising_edge(clk) then  
        rst_sync <= '0';  
    end if;  
end process;
```

A.第一部分：组合逻辑器件

1. 与门 AND

功能说明

二输入与门： $Y = A \text{ AND } B$ ，当且仅当所有输入均为'1'时输出'1'。

真值表

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

数据流描述完整代码

```
-- =====  
-- 器件：二输入与门 (AND Gate)  
-- 描述：数据流描述方式  
-- 公式：Y = A AND B  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity AND2 is  
    Port (  
        A : in  STD_LOGIC;    -- 输入A  
        B : in  STD_LOGIC;    -- 输入B  
        Y : out STD_LOGIC     -- 输出Y  
    );  
end AND2;  
  
architecture Dataflow of AND2 is  
begin  
    Y <= A and B;  
end Dataflow;
```

行为描述完整代码

```
-- =====
-- 器件：二输入与门 (AND Gate)
-- 描述：行为描述方式 (process + if-else)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity AND2_Behavioral is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end AND2_Behavioral;

architecture Behavioral of AND2_Behavioral is
begin
    process(A, B)
    begin
        if (A = '1' and B = '1') then
            Y <= '1';
        else
            Y <= '0';
        end if;
    end process;
end Behavioral;
```

结构化描述完整代码

```
-- =====  
-- 器件：二输入与门 (AND Gate)  
-- 描述：结构化描述方式（使用NAND+NOR等效实现）  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity AND2_Structural is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end AND2_Structural;  
  
architecture Structural of AND2_Structural is  
  
    -- 内部信号声明  
    signal nand_out : STD_LOGIC; -- NAND输出  
    signal not_out  : STD_LOGIC; -- NOT输出  
  
begin  
  
    -- NAND门实例  
    nand_out <= not (A and B);  
  
    -- NOT门实例  
    not_out <= not nand_out;  
  
    -- 输出赋值  
    Y <= not_out;  
  
end Structural;
```

2. 或门 OR

功能说明

二输入或门： $Y = A \text{ OR } B$ ，当任一输入为'1'时输出'1'。

真值表

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

数据流描述完整代码

```
-- =====  
-- 器件：二输入或门 (OR Gate)  
-- 描述：数据流描述方式  
-- 公式：Y = A OR B  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity OR2 is  
    Port (  
        A : in  STD_LOGIC;    -- 输入A  
        B : in  STD_LOGIC;    -- 输入B  
        Y : out STD_LOGIC     -- 输出Y  
    );  
end OR2;  
  
architecture Dataflow of OR2 is  
begin  
    Y <= A or B;  
end Dataflow;
```


行为描述完整代码

```
-- =====
-- 器件：二输入或门 (OR Gate)
-- 描述：行为描述方式 (process + case)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity OR2_Behavioral is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end OR2_Behavioral;

architecture Behavioral of OR2_Behavioral is
    -- 合并信号用于case语句
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);
begin
    inputs <= A & B;

    process(inputs)
    begin
        case inputs is
            when "00"  => Y <= '0';
            when "01"  => Y <= '1';
            when "10"  => Y <= '1';
            when "11"  => Y <= '1';
            when others => Y <= '0';
        end case;
    end process;
end Behavioral;
```

结构化描述完整代码

```
-- =====  
-- 器件：二输入或门 (OR Gate)  
-- 描述：结构化描述方式 (NOR + NOT实现)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity OR2_Structural is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end OR2_Structural;  
  
architecture Structural of OR2_Structural is  
    signal nor_out : STD_LOGIC;  
begin  
    nor_out <= A nor B;  
    Y      <= not nor_out;  
end Structural;
```

3. 非门 NOT

功能说明

非门（反相器）： $Y = \text{NOT } A$ ，输出与输入相反。

真值表

A	Y
0	1
1	0

数据流描述完整代码

```
-- =====
-- 器件：非门 (NOT Gate / Inverter)
-- 描述：数据流描述方式
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOT1 is
    Port (
        A : in  STD_LOGIC;  -- 输入A
        Y : out STD_LOGIC    -- 输出Y
    );
end NOT1;

architecture Dataflow of NOT1 is
begin
    Y <= not A;
end Dataflow;
```

行为描述完整代码

```
-- =====  
-- 器件：非门 (NOT Gate)  
-- 描述：行为描述方式 (process + if)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity NOT1_Behavioral is  
    Port (  
        A : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end NOT1_Behavioral;  
  
architecture Behavioral of NOT1_Behavioral is  
begin  
    process(A)  
    begin  
        if (A = '1') then  
            Y <= '0';  
        else  
            Y <= '1';  
        end if;  
    end process;  
end Behavioral;
```

结构化描述完整代码

```
-- =====
-- 器件：非门 (NOT Gate)
-- 描述：结构化描述方式 (NAND实现NOT)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOT1_Structural is
    Port (
        A : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end NOT1_Structural;

architecture Structural of NOT1_Structural is
begin
    Y <= A nand A;
end Structural;
```

4. 与非门 NAND

功能说明

二输入与非门： $Y = \text{NOT}(A \text{ AND } B)$ ，仅在所有输入均为'1'时输出'0'。

真值表

A	B	Y
0	0	1

A	B	Y
0	1	1
1	0	1
1	1	0

数据流描述完整代码

```
-- =====
-- 器件：二输入与非门 (NAND Gate)
-- 描述：数据流描述方式
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NAND2 is
    Port (
        A : in  STD_LOGIC;  -- 输入A
        B : in  STD_LOGIC;  -- 输入B
        Y : out STD_LOGIC    -- 输出Y
    );
end NAND2;

architecture Dataflow of NAND2 is
begin
    Y <= A nand B;
end Dataflow;
```

行为描述完整代码

```
-- =====
-- 器件：二输入与非门 (NAND Gate)
-- 描述：行为描述方式 (process + if-elsif)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NAND2_Behavioral is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end NAND2_Behavioral;

architecture Behavioral of NAND2_Behavioral is
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);
begin
    inputs <= A & B;

    process(inputs)
    begin
        if (inputs = "11") then
            Y <= '0';
        elsif (inputs = "00") then
            Y <= '1';
        elsif (inputs = "01") then
            Y <= '1';
        elsif (inputs = "10") then
            Y <= '1';
        else
            Y <= '1';
        end if;
    end process;
end Behavioral;
```

结构化描述完整代码

```
-- =====  
-- 器件：二输入与非门 (NAND Gate)  
-- 描述：结构化描述方式 (AND + NOT级联)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity NAND2_Structural is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end NAND2_Structural;  
  
architecture Structural of NAND2_Structural is  
    signal and_out : STD_LOGIC;  
begin  
    and_out <= A and B;  
    Y      <= not and_out;  
end Structural;
```

5. 或非门 NOR

功能说明

二输入或非门： $Y = \text{NOT}(A \text{ OR } B)$ ，仅在所有输入均为'0'时输出'1'。

真值表

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

数据流描述完整代码

```
-- =====
-- 器件：二输入或非门 (NOR Gate)
-- 描述：数据流描述方式
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOR2 is
    Port (
        A : in  STD_LOGIC;    -- 输入A
        B : in  STD_LOGIC;    -- 输入B
        Y : out STD_LOGIC      -- 输出Y
    );
end NOR2;

architecture Dataflow of NOR2 is
begin
    Y <= A nor B;
end Dataflow;
```

行为描述完整代码

```
-- =====  
-- 器件：二输入或非门 (NOR Gate)  
-- 描述：行为描述方式 (when-else)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity NOR2_Behavioral is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end NOR2_Behavioral;  
  
architecture Behavioral of NOR2_Behavioral is  
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);  
begin  
    inputs <= A & B;  
  
    Y <= '1' when inputs = "00" else  
        '0' when inputs = "01" else  
        '0' when inputs = "10" else  
        '0' when inputs = "11" else  
        '0';  
end Behavioral;
```

结构化描述完整代码

```
-- =====  
-- 器件：二输入或非门 (NOR Gate)  
-- 描述：结构化描述方式 (OR + NOT级联)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity NOR2_Structural is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end NOR2_Structural;  
  
architecture Structural of NOR2_Structural is  
    signal or_out : STD_LOGIC;  
begin  
    or_out <= A or B;  
    Y      <= not or_out;  
end Structural;
```

6. 异或门 XOR

功能说明

二输入异或门： $Y = A \text{ XOR } B$ ，当两个输入不相同输出'1'。

真值表

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

数据流描述完整代码

```
-- =====
-- 器件：二输入异或门 (XOR Gate)
-- 描述：数据流描述方式
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity XOR2 is
    Port (
        A : in  STD_LOGIC;  -- 输入A
        B : in  STD_LOGIC;  -- 输入B
        Y : out STD_LOGIC    -- 输出Y
    );
end XOR2;

architecture Dataflow of XOR2 is
begin
    Y <= A xor B;
end Dataflow;
```

行为描述完整代码

```
-- =====  
-- 器件：二输入异或门 (XOR Gate)  
-- 描述：行为描述方式 (with-select)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity XOR2_Behavioral is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end XOR2_Behavioral;  
  
architecture Behavioral of XOR2_Behavioral is  
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);  
begin  
    inputs <= A & B;  
  
    with inputs select  
        Y <= '0' when "00",  
            '1' when "01",  
            '1' when "10",  
            '0' when "11",  
            '0' when others;  
end Behavioral;
```

结构化描述完整代码

```
-- =====
-- 器件：二输入异或门 (XOR Gate)
-- 描述：结构化描述方式 (NAND门网络实现)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity XOR2_Structural is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end XOR2_Structural;

architecture Structural of XOR2_Structural is
    signal nand1_out : STD_LOGIC;    -- A NAND B
    signal nand2_out : STD_LOGIC;    -- A NAND (A NAND B)
    signal nand3_out : STD_LOGIC;    -- B NAND (A NAND B)
begin
    nand1_out <= A nand B;
    nand2_out <= A nand nand1_out;
    nand3_out <= B nand nand1_out;
    Y        <= nand2_out nand nand3_out;
end Structural;
```

7. 同或门 XNOR

功能说明

二输入同或门： $Y = A \text{ XNOR } B$ ，当两个输入相同时输出'1'。

真值表

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

数据流描述完整代码

```
-- =====  
-- 器件：二输入同或门 (XNOR Gate)  
-- 描述：数据流描述方式  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity XNOR2 is  
    Port (  
        A : in  STD_LOGIC;    -- 输入A  
        B : in  STD_LOGIC;    -- 输入B  
        Y : out STD_LOGIC     -- 输出Y  
    );  
end XNOR2;  
  
architecture Dataflow of XNOR2 is  
begin  
    Y <= A xnor B;  
end Dataflow;
```

行为描述完整代码

```
-- =====  
-- 器件：二输入同或门 (XNOR Gate)  
-- 描述：行为描述方式 (when-else)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity XNOR2_Behavioral is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end XNOR2_Behavioral;  
  
architecture Behavioral of XNOR2_Behavioral is  
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);  
begin  
    inputs <= A & B;  
  
    Y <= '1' when inputs = "00" else  
        '0' when inputs = "01" else  
        '0' when inputs = "10" else  
        '1' when inputs = "11" else  
        '0';  
end Behavioral;
```


结构化描述完整代码

```
-- =====  
-- 器件：二输入同或门 (XNOR Gate)  
-- 描述：结构化描述方式 (XOR + NOT级联)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity XNOR2_Structural is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end XNOR2_Structural;  
  
architecture Structural of XNOR2_Structural is  
    signal xor_out : STD_LOGIC;  
begin  
    xor_out <= A xor B;  
    Y      <= not xor_out;  
end Structural;
```

8. 三态门

功能说明

三态缓冲器：当使能信号OE = '1'时，Y = A；当OE = '0'时，Y输出高阻态'Z'。

真值表

OE	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

数据流描述完整代码

```
-- =====  
-- 器件：三态缓冲器 (Tri-State Buffer)  
-- 描述：数据流描述方式 (when-else)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity TRISTATE is  
    Port (  
        A  : in  STD_LOGIC;    -- 数据输入  
        OE : in  STD_LOGIC;    -- 输出使能（高有效）  
        Y  : out STD_LOGIC     -- 三态输出  
    );  
end TRISTATE;  
  
architecture Dataflow of TRISTATE is  
begin  
    Y <= A  when OE = '1' else 'Z';  
end Dataflow;
```

行为描述完整代码

```
-- =====  
-- 器件：三态缓冲器 (Tri-State Buffer)  
-- 描述：行为描述方式 (process + if)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity TRISTATE_Behavioral is  
    Port (  
        A  : in  STD_LOGIC;  
        OE : in  STD_LOGIC;  
        Y  : out STD_LOGIC  
    );  
end TRISTATE_Behavioral;  
  
architecture Behavioral of TRISTATE_Behavioral is  
begin  
    process(A, OE)  
    begin  
        if (OE = '1') then  
            Y <= A;  
        else  
            Y <= 'Z';  
        end if;  
    end process;  
end Behavioral;
```

结构化描述完整代码

```
-- =====  
-- 器件：四路三态缓冲器 (4-bit Tri-State Buffer)  
-- 描述：结构化描述方式 (总线驱动应用)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity TRISTATE4 is  
    Port (  
        A : in  STD_LOGIC_VECTOR(3 downto 0);    -- 4位数据输入  
        OE : in  STD_LOGIC;                        -- 输出使能  
        Y : out STD_LOGIC_VECTOR(3 downto 0)      -- 4位三态输出  
    );  
end TRISTATE4;  
  
architecture Structural of TRISTATE4 is  
begin  
    Y <= A  when OE = '1' else (others => 'Z');  
end Structural;
```

9. 半加器

功能说明

半加器实现两个1位二进制数的加法，产生和(Sum)与进位(Carry)。无低位进位输入。

真值表

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

数据流描述完整代码

```
-- =====
-- 器件：半加器 (Half Adder)
-- 描述：数据流描述方式
-- Sum = A XOR B, Carry = A AND B
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HALF_ADDER is
    Port (
        A      : in  STD_LOGIC;  -- 加数A
        B      : in  STD_LOGIC;  -- 加数B
        SUM     : out STD_LOGIC;  -- 和
        CARRY   : out STD_LOGIC   -- 进位
    );
end HALF_ADDER;

architecture Dataflow of HALF_ADDER is
begin
    SUM    <= A xor B;
    CARRY  <= A and B;
end Dataflow;
```

行为描述完整代码

```

-- =====
-- 器件：半加器 (Half Adder)
-- 描述：行为描述方式 (process + case)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HALF_ADDER_Behavioral is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        SUM     : out STD_LOGIC;
        CARRY   : out STD_LOGIC
    );
end HALF_ADDER_Behavioral;

architecture Behavioral of HALF_ADDER_Behavioral is
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);
begin
    inputs <= A & B;

    process(inputs)
    begin
        case inputs is
            when "00" =>
                SUM    <= '0';
                CARRY   <= '0';
            when "01" =>
                SUM    <= '1';
                CARRY   <= '0';
            when "10" =>
                SUM    <= '1';
                CARRY   <= '0';
            when "11" =>
                SUM    <= '0';
                CARRY   <= '1';
            when others =>
                SUM    <= '0';
        end case;
    end process;
end Behavioral;

```

```
        CARRY <= '0';  
    end case;  
end process;  
end Behavioral;
```


结构化描述完整代码

```
-- =====  
-- 器件：半加器 (Half Adder)  
-- 描述：结构化描述方式 (XOR + AND门级联)  
-- =====
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity HALF_ADDER_Structural is  
    Port (  
        A      : in  STD_LOGIC;  
        B      : in  STD_LOGIC;  
        SUM     : out STD_LOGIC;  
        CARRY  : out STD_LOGIC  
    );  
end HALF_ADDER_Structural;
```

```
architecture Structural of HALF_ADDER_Structural is
```

```
-- 元件声明
```

```
component XOR2 is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end component;
```

```
component AND2 is  
    Port (  
        A : in  STD_LOGIC;  
        B : in  STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end component;
```

```
begin
```

```
-- 异或门产生和
```

```
U1: XOR2
  Port Map (
    A => A,
    B => B,
    Y => SUM
  );

-- 与门产生进位
U2: AND2
  Port Map (
    A => A,
    B => B,
    Y => CARRY
  );

end Structural;
```

10. 全加器

功能说明

全加器实现两个1位二进制数及低位进位的加法，产生和(Sum)与进位(Carry)。

真值表

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1

A	B	Cin	Sum	Cout
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

数据流描述完整代码

```
-- =====
-- 器件：全加器 (Full Adder)
-- 描述：数据流描述方式
-- Sum = A XOR B XOR Cin
-- Cout = (A AND B) OR (A AND Cin) OR (B AND Cin)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FULL_ADDER is
    Port (
        A      : in  STD_LOGIC;    -- 加数A
        B      : in  STD_LOGIC;    -- 加数B
        CIN    : in  STD_LOGIC;    -- 低位进位输入
        SUM    : out STD_LOGIC;    -- 和
        COUT   : out STD_LOGIC     -- 进位输出
    );
end FULL_ADDER;

architecture Dataflow of FULL_ADDER is
begin
    SUM  <= A xor B xor CIN;
    COUT <= (A and B) or (A and CIN) or (B and CIN);
end Dataflow;
```

行为描述完整代码

```

-- =====
-- 器件：全加器 (Full Adder)
-- 描述：行为描述方式 (process + if-elsif)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FULL_ADDER_Behavioral is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        CIN     : in  STD_LOGIC;
        SUM     : out STD_LOGIC;
        COUT    : out STD_LOGIC
    );
end FULL_ADDER_Behavioral;

architecture Behavioral of FULL_ADDER_Behavioral is
    signal inputs : STD_LOGIC_VECTOR(2 downto 0);
begin
    inputs <= A & B & CIN;

    process(inputs)
        variable temp_sum : STD_LOGIC;
        variable temp_cout : STD_LOGIC;
    begin
        case inputs is
            when "000" =>
                temp_sum  := '0';
                temp_cout := '0';
            when "001" =>
                temp_sum  := '1';
                temp_cout := '0';
            when "010" =>
                temp_sum  := '1';
                temp_cout := '0';
            when "011" =>
                temp_sum  := '0';

```

```
        temp_cout := '1';
    when "100" =>
        temp_sum  := '1';
        temp_cout := '0';
    when "101" =>
        temp_sum  := '0';
        temp_cout := '1';
    when "110" =>
        temp_sum  := '0';
        temp_cout := '1';
    when "111" =>
        temp_sum  := '1';
        temp_cout := '1';
    when others =>
        temp_sum  := '0';
        temp_cout := '0';
    end case;
    SUM  <= temp_sum;
    COUT <= temp_cout;
end process;
end Behavioral;
```

结构化描述完整代码


```

-- =====
-- 器件：全加器 (Full Adder)
-- 描述：结构化描述方式（两个半加器级联）
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FULL_ADDER_Structural is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        CIN    : in  STD_LOGIC;
        SUM     : out STD_LOGIC;
        COUT    : out STD_LOGIC
    );
end FULL_ADDER_Structural;

architecture Structural of FULL_ADDER_Structural is

    component HALF_ADDER is
        Port (
            A      : in  STD_LOGIC;
            B      : in  STD_LOGIC;
            SUM     : out STD_LOGIC;
            CARRY   : out STD_LOGIC
        );
    end component;

    component OR2 is
        Port (
            A : in  STD_LOGIC;
            B : in  STD_LOGIC;
            Y : out STD_LOGIC
        );
    end component;

    signal sum1      : STD_LOGIC; -- 第一级半加器和
    signal carry1    : STD_LOGIC; -- 第一级半加器进位

```

```

signal carry2 : STD_LOGIC;    -- 第二级半加器进位

begin

    -- 第一级半加器: A + B
    HA1: HALF_ADDER
        Port Map (
            A      => A,
            B      => B,
            SUM     => sum1,
            CARRY  => carry1
        );

    -- 第二级半加器: sum1 + Cin
    HA2: HALF_ADDER
        Port Map (
            A      => sum1,
            B      => CIN,
            SUM     => SUM,
            CARRY  => carry2
        );

    -- 或门: 进位合并
    U1: OR2
        Port Map (
            A => carry1,
            B => carry2,
            Y => COUT
        );

end Structural;

```

11. 4位串行进位加法器

功能说明

4位串行进位（行波进位）加法器由4个全加器级联构成，进位逐级传递。

数据流描述完整代码

```
-- =====  
-- 器件：4位串行进位加法器（4-bit Ripple Carry Adder）  
-- 描述：数据流描述方式（直接使用 + 运算符）  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.STD_LOGIC_UNSIGNED.ALL;  
  
entity RCA_4BIT is  
    Port (  
        A      : in  STD_LOGIC_VECTOR(3 downto 0);    -- 4位加数A  
        B      : in  STD_LOGIC_VECTOR(3 downto 0);    -- 4位加数B  
        CIN     : in  STD_LOGIC;                      -- 进位输入  
        SUM     : out STD_LOGIC_VECTOR(3 downto 0);    -- 4位和  
        COUT    : out STD_LOGIC                      -- 进位输出  
    );  
end RCA_4BIT;  
  
architecture Dataflow of RCA_4BIT is  
    signal temp_sum : STD_LOGIC_VECTOR(4 downto 0);  
begin  
    temp_sum <= ('0' & A) + ('0' & B) + CIN;  
    SUM      <= temp_sum(3 downto 0);  
    COUT     <= temp_sum(4);  
end Dataflow;
```

行为描述完整代码

```

-- =====
-- 器件：4位串行进位加法器 (4-bit Ripple Carry Adder)
-- 描述：行为描述方式 (process + case + 中间进位)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4BIT_Behavioral is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        CIN     : in  STD_LOGIC;
        SUM     : out STD_LOGIC_VECTOR(3 downto 0);
        COUT    : out STD_LOGIC
    );
end RCA_4BIT_Behavioral;

architecture Behavioral of RCA_4BIT_Behavioral is
begin
    process(A, B, CIN)
        variable carry : STD_LOGIC_VECTOR(4 downto 0);
        variable temp_a : STD_LOGIC_VECTOR(3 downto 0);
        variable temp_b : STD_LOGIC_VECTOR(3 downto 0);
        variable temp_sum : STD_LOGIC_VECTOR(3 downto 0);
    begin
        carry(0) := CIN;
        temp_a   := A;
        temp_b   := B;

        for i in 0 to 3 loop
            temp_sum(i) := temp_a(i) xor temp_b(i) xor carry(i);
            carry(i+1) := (temp_a(i) and temp_b(i)) or
                           (temp_a(i) and carry(i)) or
                           (temp_b(i) and carry(i));
        end loop;

        SUM <= temp_sum;
        COUT <= carry(4);
    end process;
end Behavioral;

```

```
    end process;  
end Behavioral;
```

结构化描述完整代码

```

-- =====
-- 器件：4位串行进位加法器（4-bit Ripple Carry Adder）
-- 描述：结构化描述方式（4个全加器级联）
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4BIT_Structural is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);
        B    : in  STD_LOGIC_VECTOR(3 downto 0);
        CIN   : in  STD_LOGIC;
        SUM   : out STD_LOGIC_VECTOR(3 downto 0);
        COUT  : out STD_LOGIC
    );
end RCA_4BIT_Structural;

architecture Structural of RCA_4BIT_Structural is

    component FULL_ADDER is
        Port (
            A    : in  STD_LOGIC;
            B    : in  STD_LOGIC;
            CIN   : in  STD_LOGIC;
            SUM   : out STD_LOGIC;
            COUT  : out STD_LOGIC
        );
    end component;

    signal carry : STD_LOGIC_VECTOR(3 downto 0); -- 内部进位链

begin

    FA0: FULL_ADDER
        Port Map (
            A    => A(0),
            B    => B(0),
            CIN   => CIN,

```



```
    SUM => SUM(0),  
    COUT => carry(0)  
);
```

FA1: FULL_ADDER

```
Port Map (  
    A    => A(1),  
    B    => B(1),  
    CIN  => carry(0),  
    SUM  => SUM(1),  
    COUT => carry(1)  
);
```

FA2: FULL_ADDER

```
Port Map (  
    A    => A(2),  
    B    => B(2),  
    CIN  => carry(1),  
    SUM  => SUM(2),  
    COUT => carry(2)  
);
```

FA3: FULL_ADDER

```
Port Map (  
    A    => A(3),  
    B    => B(3),  
    CIN  => carry(2),  
    SUM  => SUM(3),  
    COUT => COUT  
);
```

```
end Structural;
```

12. 超前进位加法器 CLA

功能说明

超前进位加法器 (Carry Look-Ahead Adder) 通过并行计算进位信号来提高速度，避免串行进位的逐级传递延迟。

数据流描述完整代码

```

-- =====
-- 器件：4位超前进位加法器 (4-bit CLA)
-- 描述：数据流描述方式
-- 进位生成：Gi = Ai AND Bi
-- 进位传播：Pi = Ai XOR Bi
-- 进位输出：Ci+1 = Gi OR (Pi AND Ci)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity CLA_4BIT is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);    -- 4位加数A
        B    : in  STD_LOGIC_VECTOR(3 downto 0);    -- 4位加数B
        CIN   : in  STD_LOGIC;                      -- 进位输入
        SUM   : out STD_LOGIC_VECTOR(3 downto 0);    -- 4位和
        COUT  : out STD_LOGIC                       -- 进位输出
    );
end CLA_4BIT;

architecture Dataflow of CLA_4BIT is
    signal G : STD_LOGIC_VECTOR(3 downto 0);    -- 进位生成信号
    signal P : STD_LOGIC_VECTOR(3 downto 0);    -- 进位传播信号
    signal C : STD_LOGIC_VECTOR(4 downto 0);    -- 内部进位
begin
    C(0) <= CIN;

    G(0) <= A(0) and B(0);
    G(1) <= A(1) and B(1);
    G(2) <= A(2) and B(2);
    G(3) <= A(3) and B(3);

    P(0) <= A(0) xor B(0);
    P(1) <= A(1) xor B(1);
    P(2) <= A(2) xor B(2);
    P(3) <= A(3) xor B(3);

    -- 超前进位逻辑

```

```
C(1) <= G(0) or (P(0) and C(0));
C(2) <= G(1) or (P(1) and G(0)) or (P(1) and P(0) and C(0));
C(3) <= G(2) or (P(2) and G(1)) or (P(2) and P(1) and G(0))
      or (P(2) and P(1) and P(0) and C(0));
C(4) <= G(3) or (P(3) and G(2)) or (P(3) and P(2) and G(1))
      or (P(3) and P(2) and P(1) and G(0))
      or (P(3) and P(2) and P(1) and P(0) and C(0));
```

```
SUM(0) <= P(0) xor C(0);
SUM(1) <= P(1) xor C(1);
SUM(2) <= P(2) xor C(2);
SUM(3) <= P(3) xor C(3);
```

```
COUT <= C(4);
```

```
end Dataflow;
```

行为描述完整代码

```

-- =====
-- 器件：4位超前进位加法器 (4-bit CLA)
-- 描述：行为描述方式 (process)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity CLA_4BIT_Behavioral is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);
        B    : in  STD_LOGIC_VECTOR(3 downto 0);
        CIN   : in  STD_LOGIC;
        SUM   : out STD_LOGIC_VECTOR(3 downto 0);
        COUT  : out STD_LOGIC
    );
end CLA_4BIT_Behavioral;

architecture Behavioral of CLA_4BIT_Behavioral is
begin
    process(A, B, CIN)
        variable G : STD_LOGIC_VECTOR(3 downto 0);
        variable P : STD_LOGIC_VECTOR(3 downto 0);
        variable C : STD_LOGIC_VECTOR(4 downto 0);
    begin
        C(0) := CIN;

        for i in 0 to 3 loop
            G(i) := A(i) and B(i);
            P(i) := A(i) xor B(i);
        end loop;

        C(1) := G(0) or (P(0) and C(0));
        C(2) := G(1) or (P(1) and G(0)) or (P(1) and P(0) and C(0));
        C(3) := G(2) or (P(2) and G(1)) or (P(2) and P(1) and G(0))
            or (P(2) and P(1) and P(0) and C(0));
        C(4) := G(3) or (P(3) and G(2)) or (P(3) and P(2) and G(1))
            or (P(3) and P(2) and P(1) and G(0))
            or (P(3) and P(2) and P(1) and P(0) and C(0));
    end process;
end Behavioral;

```

```
    for i in 0 to 3 loop
        SUM(i) <= P(i) xor C(i);
    end loop;

    COUT <= C(4);
end process;
end Behavioral;
```


结构化描述完整代码

```

-- =====
-- 器件：4位超前进位加法器（4-bit CLA）
-- 描述：结构化描述方式（CLA模块 + 全加器阵列）
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity CLA_4BIT_Structural is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);
        B    : in  STD_LOGIC_VECTOR(3 downto 0);
        CIN   : in  STD_LOGIC;
        SUM   : out STD_LOGIC_VECTOR(3 downto 0);
        COUT  : out STD_LOGIC
    );
end CLA_4BIT_Structural;

architecture Structural of CLA_4BIT_Structural is

    component FULL_ADDER is
        Port (
            A    : in  STD_LOGIC;
            B    : in  STD_LOGIC;
            CIN   : in  STD_LOGIC;
            SUM   : out STD_LOGIC;
            COUT  : out STD_LOGIC
        );
    end component;

    signal G : STD_LOGIC_VECTOR(3 downto 0);
    signal P : STD_LOGIC_VECTOR(3 downto 0);
    signal C : STD_LOGIC_VECTOR(4 downto 0);

begin
    C(0) <= CIN;

    G(0) <= A(0) and B(0);
    G(1) <= A(1) and B(1);

```

```
G(2) <= A(2) and B(2);
G(3) <= A(3) and B(3);

P(0) <= A(0) xor B(0);
P(1) <= A(1) xor B(1);
P(2) <= A(2) xor B(2);
P(3) <= A(3) xor B(3);

C(1) <= G(0) or (P(0) and C(0));
C(2) <= G(1) or (P(1) and G(0)) or (P(1) and P(0) and C(0));
C(3) <= G(2) or (P(2) and G(1)) or (P(2) and P(1) and G(0))
      or (P(2) and P(1) and P(0) and C(0));
C(4) <= G(3) or (P(3) and G(2)) or (P(3) and P(2) and G(1))
      or (P(3) and P(2) and P(1) and G(0))
      or (P(3) and P(2) and P(1) and P(0) and C(0));

FA0: FULL_ADDER Port Map (A => A(0), B => B(0), CIN => C(0), SUM => SUM(0), COUT => open);
FA1: FULL_ADDER Port Map (A => A(1), B => B(1), CIN => C(1), SUM => SUM(1), COUT => open);
FA2: FULL_ADDER Port Map (A => A(2), B => B(2), CIN => C(2), SUM => SUM(2), COUT => open);
FA3: FULL_ADDER Port Map (A => A(3), B => B(3), CIN => C(3), SUM => SUM(3), COUT => open);

COUT <= C(4);
end Structural;
```

13. BCD加法器

功能说明

BCD加法器实现两个4位BCD码的加法。当结果超过9时加6修正。

真值表（修正逻辑）

二进制和(5位)	修正
0~9	不加
10~19	+6

数据流描述完整代码

```

-- =====
-- 器件：1位BCD加法器 (BCD Adder)
-- 描述：数据流描述方式
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BCD_ADDER is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);    -- BCD加数A
        B      : in  STD_LOGIC_VECTOR(3 downto 0);    -- BCD加数B
        CIN     : in  STD_LOGIC;                      -- 进位输入
        SUM     : out STD_LOGIC_VECTOR(3 downto 0);    -- BCD和输出
        COUT    : out STD_LOGIC                       -- 进位输出
    );
end BCD_ADDER;

architecture Dataflow of BCD_ADDER is
    signal binary_sum : STD_LOGIC_VECTOR(4 downto 0);    -- 5位二进制和
    signal bcd_sum     : STD_LOGIC_VECTOR(4 downto 0);    -- BCD修正后的和
    signal correction : STD_LOGIC;                      -- 修正使能
begin
    -- 第一步：二进制加法
    binary_sum <= ('0' & A) + ('0' & B) + CIN;

    -- 修正条件：和 > 9 或 产生进位
    correction <= '1' when (binary_sum > 9) else '0';

    -- 第二步：加6修正
    bcd_sum <= binary_sum when (correction = '0') else
        binary_sum + "00110";

    SUM  <= bcd_sum(3 downto 0);
    COUT <= correction;
end Dataflow;

```

行为描述完整代码

```
-- =====
-- 器件：1位BCD加法器 (BCD Adder)
-- 描述：行为描述方式 (process)
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BCD_ADDER_Behavioral is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        CIN     : in  STD_LOGIC;
        SUM     : out STD_LOGIC_VECTOR(3 downto 0);
        COUT    : out STD_LOGIC
    );
end BCD_ADDER_Behavioral;

architecture Behavioral of BCD_ADDER_Behavioral is
begin
    process(A, B, CIN)
        variable temp : STD_LOGIC_VECTOR(4 downto 0);
    begin
        temp := ('0' & A) + ('0' & B) + CIN;

        if (temp > 9) then
            temp := temp + "00110";
            COUT <= '1';
        else
            COUT <= '0';
        end if;

        SUM <= temp(3 downto 0);
    end process;
end Behavioral;
```

结构化描述完整代码


```

-- =====
-- 器件：1位BCD加法器（BCD Adder）
-- 描述：结构化描述方式（4位加法器 + 修正逻辑）
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BCD_ADDER_Structural is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        CIN     : in  STD_LOGIC;
        SUM     : out STD_LOGIC_VECTOR(3 downto 0);
        COUT    : out STD_LOGIC
    );
end BCD_ADDER_Structural;

architecture Structural of BCD_ADDER_Structural is

    component RCA_4BIT is
        Port (
            A      : in  STD_LOGIC_VECTOR(3 downto 0);
            B      : in  STD_LOGIC_VECTOR(3 downto 0);
            CIN     : in  STD_LOGIC;
            SUM     : out STD_LOGIC_VECTOR(3 downto 0);
            COUT    : out STD_LOGIC
        );
    end component;

    signal binary_sum  : STD_LOGIC_VECTOR(3 downto 0);
    signal carry1      : STD_LOGIC;
    signal correction  : STD_LOGIC;
    signal bcd_sum     : STD_LOGIC_VECTOR(3 downto 0);
    signal add6_input  : STD_LOGIC_VECTOR(3 downto 0);

begin
    ADDER1: RCA_4BIT

```

```
Port Map (A => A, B => B, CIN => CIN, SUM => binary_sum, COUT => carry1);

correction <= carry1 or (binary_sum(3) and binary_sum(2))
              or (binary_sum(3) and binary_sum(1));

add6_input <= "0110" when correction = '1' else "0000";

ADDER2: RCA_4BIT
Port Map (A => binary_sum, B => add6_input, CIN => '0', SUM => bcd_sum, COUT => open);

SUM <= bcd_sum;
COUT <= correction;
end Structural;
```

14. 串行加法器

功能说明

串行加法器逐位相加，使用移位寄存器和单个全加器，N位需要N个时钟周期完成。

数据流描述完整代码

```

-- =====
-- 器件：4位串行加法器 (Serial Adder)
-- 描述：状态机控制 + 移位寄存器
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SERIAL_ADDER is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0); -- 4位加数A
        B      : in  STD_LOGIC_VECTOR(3 downto 0); -- 4位加数B
        CLK     : in  STD_LOGIC;                    -- 时钟
        RST     : in  STD_LOGIC;                    -- 复位
        START   : in  STD_LOGIC;                    -- 启动信号
        SUM     : out STD_LOGIC_VECTOR(4 downto 0); -- 5位和（含进位）
        DONE    : out STD_LOGIC                     -- 完成标志
    );
end SERIAL_ADDER;

architecture Dataflow of SERIAL_ADDER is
    signal shift_a  : STD_LOGIC_VECTOR(3 downto 0);
    signal shift_b  : STD_LOGIC_VECTOR(3 downto 0);
    signal shift_sum : STD_LOGIC_VECTOR(4 downto 0);
    signal carry     : STD_LOGIC;
    signal bit_cnt   : INTEGER range 0 to 4;
    signal busy      : STD_LOGIC;
begin

    process(CLK, RST)
    begin
        if (RST = '1') then
            shift_a  <= (others => '0');
            shift_b  <= (others => '0');
            shift_sum <= (others => '0');
            carry     <= '0';
            bit_cnt   <= 0;
            busy      <= '0';
            DONE      <= '0';
        end if;
    end process;
end Dataflow;

```

```

SUM      <= (others => '0');

elsif rising_edge(CLK) then
    if (START = '1' and busy = '0') then
        shift_a <= A;
        shift_b <= B;
        carry    <= '0';
        bit_cnt  <= 0;
        busy     <= '1';
        DONE     <= '0';
        shift_sum <= (others => '0');

    elsif (busy = '1') then
        if (bit_cnt < 4) then
            shift_sum(bit_cnt) <= shift_a(0) xor shift_b(0) xor carry;
            carry <= (shift_a(0) and shift_b(0)) or
                    (shift_a(0) and carry) or
                    (shift_b(0) and carry);
            shift_a <= '0' & shift_a(3 downto 1);
            shift_b <= '0' & shift_b(3 downto 1);
            bit_cnt <= bit_cnt + 1;
        else
            shift_sum(4) <= carry;
            SUM <= shift_sum;
            DONE <= '1';
            busy <= '0';
        end if;
    end if;
end if;
end process;

end Dataflow;

```

行为描述完整代码

```

-- =====
-- 器件：4位串行加法器 (Serial Adder)
-- 描述：行为描述方式（简化状态机）
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SERIAL_ADDER_Behavioral is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        CLK     : in  STD_LOGIC;
        RST     : in  STD_LOGIC;
        START   : in  STD_LOGIC;
        SUM     : out STD_LOGIC_VECTOR(4 downto 0);
        DONE    : out STD_LOGIC
    );
end SERIAL_ADDER_Behavioral;

architecture Behavioral of SERIAL_ADDER_Behavioral is
    type state_type is (IDLE, LOAD, COMPUTE, FINISH);
    signal state : state_type;
    signal reg_a  : STD_LOGIC_VECTOR(3 downto 0);
    signal reg_b  : STD_LOGIC_VECTOR(3 downto 0);
    signal result : STD_LOGIC_VECTOR(4 downto 0);
    signal carry  : STD_LOGIC;
    signal count  : INTEGER range 0 to 3;
begin

    process(CLK, RST)
    begin
        if (RST = '1') then
            state <= IDLE;
            reg_a <= (others => '0');
            reg_b <= (others => '0');
            result <= (others => '0');
            carry <= '0';
            count <= 0;
        end if;
    end process;
end Behavioral;

```

```

DONE    <= '0';
SUM     <= (others => '0');

elsif rising_edge(CLK) then
    case state is

        when IDLE =>
            DONE <= '0';
            if (START = '1') then
                reg_a <= A;
                reg_b <= B;
                carry <= '0';
                count <= 0;
                result <= (others => '0');
                state <= LOAD;
            end if;

        when LOAD =>
            state <= COMPUTE;

        when COMPUTE =>
            result(count) <= reg_a(0) xor reg_b(0) xor carry;
            carry <= (reg_a(0) and reg_b(0)) or
                (reg_a(0) and carry) or
                (reg_b(0) and carry);
            reg_a <= '0' & reg_a(3 downto 1);
            reg_b <= '0' & reg_b(3 downto 1);

            if (count = 3) then
                result(4) <= carry;
                state <= FINISH;
            else
                count <= count + 1;
            end if;

        when FINISH =>
            SUM <= result;
            DONE <= '1';
            state <= IDLE;
    end case;
end if;

```



```
        when others =>
            state <= IDLE;

        end case;
    end if;
end process;

end Behavioral;
```

15. 半减器

功能说明

半减器实现两个1位二进制数的减法：差(Diff) = A - B，借位(Borrow) = (NOT A) AND B。

真值表

A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HALF_SUBTRACTOR is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        DIFF   : out STD_LOGIC;
        BORROW : out STD_LOGIC
    );
end HALF_SUBTRACTOR;

architecture Dataflow of HALF_SUBTRACTOR is
begin
    DIFF   <= A xor B;
    BORROW <= (not A) and B;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HALF_SUBTRACTOR_Behavioral is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        DIFF   : out STD_LOGIC;
        BORROW : out STD_LOGIC
    );
end HALF_SUBTRACTOR_Behavioral;

architecture Behavioral of HALF_SUBTRACTOR_Behavioral is
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);
begin
    inputs <= A & B;
    process(inputs)
    begin
        case inputs is
            when "00" => DIFF <= '0'; BORROW <= '0';
            when "01" => DIFF <= '1'; BORROW <= '1';
            when "10" => DIFF <= '1'; BORROW <= '0';
            when "11" => DIFF <= '0'; BORROW <= '0';
            when others => DIFF <= '0'; BORROW <= '0';
        end case;
    end process;
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HALF_SUBTRACTOR_Structural is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        DIFF   : out STD_LOGIC;
        BORROW : out STD_LOGIC
    );
end HALF_SUBTRACTOR_Structural;

architecture Structural of HALF_SUBTRACTOR_Structural is
    signal not_a : STD_LOGIC;
begin
    not_a <= not A;
    DIFF  <= A xor B;
    BORROW <= not_a and B;
end Structural;
```

16. 全减器

功能说明

全减器实现A-B-Bin，产生差(Diff)和借位(Bout)。

真值表

A	B	Bin	Diff	Bout
0	0	0	0	0

A	B	Bin	Diff	Bout
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FULL_SUBTRACTOR is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        BIN    : in  STD_LOGIC;
        DIFF   : out STD_LOGIC;
        BOUT   : out STD_LOGIC
    );
end FULL_SUBTRACTOR;

architecture Dataflow of FULL_SUBTRACTOR is
begin
    DIFF <= A xor B xor BIN;
    BOUT <= ((not A) and B) or ((not A) and BIN) or (B and BIN);
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FULL_SUBTRACTOR_Behavioral is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        BIN    : in  STD_LOGIC;
        DIFF   : out STD_LOGIC;
        BOUT   : out STD_LOGIC
    );
end FULL_SUBTRACTOR_Behavioral;

architecture Behavioral of FULL_SUBTRACTOR_Behavioral is
    signal inputs : STD_LOGIC_VECTOR(2 downto 0);
begin
    inputs <= A & B & BIN;
    process(inputs)
    begin
        case inputs is
            when "000" => DIFF <= '0'; BOUT <= '0';
            when "001" => DIFF <= '1'; BOUT <= '1';
            when "010" => DIFF <= '1'; BOUT <= '1';
            when "011" => DIFF <= '0'; BOUT <= '1';
            when "100" => DIFF <= '1'; BOUT <= '0';
            when "101" => DIFF <= '0'; BOUT <= '0';
            when "110" => DIFF <= '0'; BOUT <= '0';
            when "111" => DIFF <= '1'; BOUT <= '1';
            when others => DIFF <= '0'; BOUT <= '0';
        end case;
    end process;
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FULL_SUBTRACTOR_Structural is
    Port (
        A      : in  STD_LOGIC;
        B      : in  STD_LOGIC;
        BIN     : in  STD_LOGIC;
        DIFF    : out STD_LOGIC;
        BOUT    : out STD_LOGIC
    );
end FULL_SUBTRACTOR_Structural;

architecture Structural of FULL_SUBTRACTOR_Structural is

    component HALF_SUBTRACTOR is
        Port (
            A      : in  STD_LOGIC;
            B      : in  STD_LOGIC;
            DIFF    : out STD_LOGIC;
            BORROW  : out STD_LOGIC
        );
    end component;

    signal diff1    : STD_LOGIC;
    signal borrow1  : STD_LOGIC;
    signal borrow2  : STD_LOGIC;

begin
    HS1: HALF_SUBTRACTOR Port Map (A => A, B => B, DIFF => diff1, BORROW => borrow1);
    HS2: HALF_SUBTRACTOR Port Map (A => diff1, B => BIN, DIFF => DIFF, BORROW => borrow2);
    BOUT <= borrow1 or borrow2;
end Structural;
```

17. BCD减法器

功能说明

BCD减法器实现两个4位BCD码的减法，采用10的补码方式。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BCD_SUBTRACTOR is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        BIN     : in  STD_LOGIC;
        DIFF    : out STD_LOGIC_VECTOR(3 downto 0);
        BOUT     : out STD_LOGIC
    );
end BCD_SUBTRACTOR;

architecture Dataflow of BCD_SUBTRACTOR is
    signal comp_9      : STD_LOGIC_VECTOR(3 downto 0);
    signal add_result   : STD_LOGIC_VECTOR(4 downto 0);
    signal no_borrow    : STD_LOGIC;
begin
    comp_9 <= "1001" - B;
    add_result <= ('0' & A) + ('0' & comp_9) + (not BIN);
    no_borrow <= '1' when (add_result > 9) else '0';

    DIFF <= add_result(3 downto 0) when (no_borrow = '1') else
            (add_result(3 downto 0) + "0110");
    BOUT <= not no_borrow;
end Dataflow;
```


行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BCD_SUBTRACTOR_Behavioral is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        BIN    : in  STD_LOGIC;
        DIFF   : out STD_LOGIC_VECTOR(3 downto 0);
        BOUT   : out STD_LOGIC
    );
end BCD_SUBTRACTOR_Behavioral;

architecture Behavioral of BCD_SUBTRACTOR_Behavioral is
begin
    process(A, B, BIN)
        variable comp_9      : STD_LOGIC_VECTOR(3 downto 0);
        variable add_result : STD_LOGIC_VECTOR(4 downto 0);
    begin
        comp_9 := "1001" - B;
        add_result := ('0' & A) + ('0' & comp_9) + (not BIN);

        if (add_result > 9) then
            DIFF <= add_result(3 downto 0);
            BOUT <= '0';
        else
            DIFF <= add_result(3 downto 0) + "0110";
            BOUT <= '1';
        end if;
    end process;
end Behavioral;
```

18. 1位比较器

功能说明

1位比较器比较两个1位二进制数A和B，产生大于(G)、等于(E)、小于(L)三种输出。

真值表

A	B	G	E	L
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_1BIT is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        G : out STD_LOGIC;
        E : out STD_LOGIC;
        L : out STD_LOGIC
    );
end COMPARATOR_1BIT;

architecture Dataflow of COMPARATOR_1BIT is
begin
    G <= A and (not B);
    E <= A xnor B;
    L <= (not A) and B;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_1BIT_Behavioral is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        G : out STD_LOGIC;
        E : out STD_LOGIC;
        L : out STD_LOGIC
    );
end COMPARATOR_1BIT_Behavioral;

architecture Behavioral of COMPARATOR_1BIT_Behavioral is
    signal inputs : STD_LOGIC_VECTOR(1 downto 0);
begin
    inputs <= A & B;
    process(inputs)
    begin
        if (inputs = "10") then
            G <= '1'; E <= '0'; L <= '0';
        elsif (inputs = "01") then
            G <= '0'; E <= '0'; L <= '1';
        else
            G <= '0'; E <= '1'; L <= '0';
        end if;
    end process;
end Behavioral;
```

19. 4位比较器

功能说明

4位比较器比较两个4位二进制数，从高位到低位逐位比较。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_4BIT is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);
        B    : in  STD_LOGIC_VECTOR(3 downto 0);
        G_IN : in  STD_LOGIC;
        E_IN : in  STD_LOGIC;
        L_IN : in  STD_LOGIC;
        G     : out STD_LOGIC;
        E     : out STD_LOGIC;
        L     : out STD_LOGIC
    );
end COMPARATOR_4BIT;

architecture Dataflow of COMPARATOR_4BIT is
begin
    process(A, B, G_IN, E_IN, L_IN)
    begin
        if (A > B) then
            G <= '1'; E <= '0'; L <= '0';
        elsif (A < B) then
            G <= '0'; E <= '0'; L <= '1';
        else
            G <= G_IN; E <= E_IN; L <= L_IN;
        end if;
    end process;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_4BIT_Behavioral is
    Port (
        A : in  STD_LOGIC_VECTOR(3 downto 0);
        B : in  STD_LOGIC_VECTOR(3 downto 0);
        G : out STD_LOGIC;
        E : out STD_LOGIC;
        L : out STD_LOGIC
    );
end COMPARATOR_4BIT_Behavioral;

architecture Behavioral of COMPARATOR_4BIT_Behavioral is
begin
    G <= '1' when (A > B) else '0';
    E <= '1' when (A = B) else '0';
    L <= '1' when (A < B) else '0';
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_4BIT_Structural is
    Port (
        A : in  STD_LOGIC_VECTOR(3 downto 0);
        B : in  STD_LOGIC_VECTOR(3 downto 0);
        G : out STD_LOGIC;
        E : out STD_LOGIC;
        L : out STD_LOGIC
    );
end COMPARATOR_4BIT_Structural;

architecture Structural of COMPARATOR_4BIT_Structural is
    component COMPARATOR_1BIT is
        Port (A : in STD_LOGIC; B : in STD_LOGIC; G : out STD_LOGIC; E : out STD_LOGIC; L : out STD_LOGIC);
    end component;

    signal g_arr : STD_LOGIC_VECTOR(3 downto 0);
    signal e_arr : STD_LOGIC_VECTOR(3 downto 0);
    signal l_arr : STD_LOGIC_VECTOR(3 downto 0);
begin
    C0: COMPARATOR_1BIT Port Map (A(0), B(0), g_arr(0), e_arr(0), l_arr(0));
    C1: COMPARATOR_1BIT Port Map (A(1), B(1), g_arr(1), e_arr(1), l_arr(1));
    C2: COMPARATOR_1BIT Port Map (A(2), B(2), g_arr(2), e_arr(2), l_arr(2));
    C3: COMPARATOR_1BIT Port Map (A(3), B(3), g_arr(3), e_arr(3), l_arr(3));

    G <= g_arr(3) or (e_arr(3) and g_arr(2)) or (e_arr(3) and e_arr(2) and g_arr(1))
        or (e_arr(3) and e_arr(2) and e_arr(1) and g_arr(0));
    L <= l_arr(3) or (e_arr(3) and l_arr(2)) or (e_arr(3) and e_arr(2) and l_arr(1))
        or (e_arr(3) and e_arr(2) and e_arr(1) and l_arr(0));
    E <= e_arr(3) and e_arr(2) and e_arr(1) and e_arr(0);
end Structural;
```


20. 8位比较器

功能说明

8位比较器比较两个8位二进制数，生成大于、等于、小于输出。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_8BIT is
    Port (
        A : in  STD_LOGIC_VECTOR(7 downto 0);
        B : in  STD_LOGIC_VECTOR(7 downto 0);
        G : out STD_LOGIC;
        E : out STD_LOGIC;
        L : out STD_LOGIC
    );
end COMPARATOR_8BIT;

architecture Dataflow of COMPARATOR_8BIT is
begin
    G <= '1' when (A > B) else '0';
    E <= '1' when (A = B) else '0';
    L <= '1' when (A < B) else '0';
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_8BIT_Behavioral is
    Port (
        A : in  STD_LOGIC_VECTOR(7 downto 0);
        B : in  STD_LOGIC_VECTOR(7 downto 0);
        G : out STD_LOGIC;
        E : out STD_LOGIC;
        L : out STD_LOGIC
    );
end COMPARATOR_8BIT_Behavioral;

architecture Behavioral of COMPARATOR_8BIT_Behavioral is
begin
    process(A, B)
    begin
        G <= '0'; E <= '0'; L <= '0';
        if (A > B) then
            G <= '1';
        elsif (A < B) then
            L <= '1';
        else
            E <= '1';
        end if;
    end process;
end Behavioral;
```

结构化描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity COMPARATOR_8BIT_Structural is
    Port (
        A : in  STD_LOGIC_VECTOR(7 downto 0);
        B : in  STD_LOGIC_VECTOR(7 downto 0);
        G : out STD_LOGIC;
        E : out STD_LOGIC;
        L : out STD_LOGIC
    );
end COMPARATOR_8BIT_Structural;

architecture Structural of COMPARATOR_8BIT_Structural is

    component COMPARATOR_4BIT is
        Port (
            A      : in  STD_LOGIC_VECTOR(3 downto 0);
            B      : in  STD_LOGIC_VECTOR(3 downto 0);
            G_IN   : in  STD_LOGIC;
            E_IN   : in  STD_LOGIC;
            L_IN   : in  STD_LOGIC;
            G      : out STD_LOGIC;
            E      : out STD_LOGIC;
            L      : out STD_LOGIC
        );
    end component;

    signal g_low, e_low, l_low : STD_LOGIC;

begin
    CMP_LOW: COMPARATOR_4BIT
        Port Map (A(3 downto 0), B(3 downto 0), '0', '1', '0', g_low, e_low, l_low);

    CMP_HIGH: COMPARATOR_4BIT
        Port Map (A(7 downto 4), B(7 downto 4), g_low, e_low, l_low, G, E, L);
end Structural;

```

21. 2选1MUX

功能说明

2选1多路选择器：根据选择信号S选择A或B输出。Y = A when S=0 else B。

真值表

S	Y
0	A
1	B

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2T01 is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        S : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end MUX_2T01;

architecture Dataflow of MUX_2T01 is
begin
    Y <= A when S = '0' else B;
end Dataflow;
```

行为描述完整代码 (process + if)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2T01_Behavioral is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        S : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end MUX_2T01_Behavioral;

architecture Behavioral of MUX_2T01_Behavioral is
begin
    process(A, B, S)
    begin
        if (S = '0') then
            Y <= A;
        else
            Y <= B;
        end if;
    end process;
end Behavioral;
```

行为描述完整代码 (with-select)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2T01_WithSelect is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        S : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end MUX_2T01_WithSelect;

architecture Behavioral of MUX_2T01_WithSelect is
begin
    with S select
        Y <= A when '0',
            B when '1',
            '0' when others;
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2T01_Structural is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        S : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end MUX_2T01_Structural;

architecture Structural of MUX_2T01_Structural is
    signal s_not    : STD_LOGIC;
    signal and1_out : STD_LOGIC;
    signal and2_out : STD_LOGIC;
begin
    s_not    <= not S;
    and1_out <= A and s_not;
    and2_out <= B and S;
    Y        <= and1_out or and2_out;
end Structural;
```

22. 4选1MUX

功能说明

4选1多路选择器：根据2位选择信号S从4个输入中选1个输出。

真值表

S1	S0	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_4T01 is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        S : in  STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC
    );
end MUX_4T01;

architecture Dataflow of MUX_4T01 is
begin
    Y <= D(0) when S = "00" else
        D(1) when S = "01" else
        D(2) when S = "10" else
        D(3) when S = "11" else
        '0';
end Dataflow;
```

行为描述完整代码 (process + case)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_4T01_Behavioral is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        S : in  STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC
    );
end MUX_4T01_Behavioral;

architecture Behavioral of MUX_4T01_Behavioral is
begin
    process(D, S)
    begin
        case S is
            when "00"  => Y <= D(0);
            when "01"  => Y <= D(1);
            when "10"  => Y <= D(2);
            when "11"  => Y <= D(3);
            when others => Y <= '0';
        end case;
    end process;
end Behavioral;
```

行为描述完整代码 (with-select)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_4T01_WithSelect is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        S : in  STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC
    );
end MUX_4T01_WithSelect;

architecture Behavioral of MUX_4T01_WithSelect is
begin
    with S select
        Y <= D(0) when "00",
            D(1) when "01",
            D(2) when "10",
            D(3) when "11",
            '0'  when others;
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_4T01_Structural is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        S : in  STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC
    );
end MUX_4T01_Structural;

architecture Structural of MUX_4T01_Structural is

    component MUX_2T01 is
        Port (A : in STD_LOGIC; B : in STD_LOGIC; S : in STD_LOGIC; Y : out STD_LOGIC);
    end component;

    signal mux0_out : STD_LOGIC;
    signal mux1_out : STD_LOGIC;

begin
    MUX0: MUX_2T01 Port Map (D(0), D(1), S(0), mux0_out);
    MUX1: MUX_2T01 Port Map (D(2), D(3), S(0), mux1_out);
    MUX_FINAL: MUX_2T01 Port Map (mux0_out, mux1_out, S(1), Y);
end Structural;
```

23. 8选1MUX

功能说明

8选1多路选择器：根据3位选择信号S从8个输入中选1个输出。

真值表

S2	S1	S0	Y
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	0	0	D4
1	0	1	D5
1	1	0	D6
1	1	1	D7

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_8T01 is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC
    );
end MUX_8T01;

architecture Dataflow of MUX_8T01 is
begin
    Y <= D(0) when S = "000" else
        D(1) when S = "001" else
        D(2) when S = "010" else
        D(3) when S = "011" else
        D(4) when S = "100" else
        D(5) when S = "101" else
        D(6) when S = "110" else
        D(7) when S = "111" else
        '0';
end Dataflow;
```

行为描述完整代码 (process + case)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_8T01_Behavioral is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC
    );
end MUX_8T01_Behavioral;

architecture Behavioral of MUX_8T01_Behavioral is
begin
    process(D, S)
    begin
        case S is
            when "000" => Y <= D(0);
            when "001" => Y <= D(1);
            when "010" => Y <= D(2);
            when "011" => Y <= D(3);
            when "100" => Y <= D(4);
            when "101" => Y <= D(5);
            when "110" => Y <= D(6);
            when "111" => Y <= D(7);
            when others => Y <= '0';
        end case;
    end process;
end Behavioral;
```

行为描述完整代码 (if-elsif)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_8T01_IfElsif is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC
    );
end MUX_8T01_IfElsif;

architecture Behavioral of MUX_8T01_IfElsif is
begin
    process(D, S)
    begin
        if (S = "000") then
            Y <= D(0);
        elsif (S = "001") then
            Y <= D(1);
        elsif (S = "010") then
            Y <= D(2);
        elsif (S = "011") then
            Y <= D(3);
        elsif (S = "100") then
            Y <= D(4);
        elsif (S = "101") then
            Y <= D(5);
        elsif (S = "110") then
            Y <= D(6);
        elsif (S = "111") then
            Y <= D(7);
        else
            Y <= '0';
        end if;
    end process;
end Behavioral;
```


结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_8T01_Structural is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC
    );
end MUX_8T01_Structural;

architecture Structural of MUX_8T01_Structural is

    component MUX_4T01 is
        Port (D : in STD_LOGIC_VECTOR(3 downto 0); S : in STD_LOGIC_VECTOR(1 downto 0); Y : out STD_LOGIC);
    end component;

    component MUX_2T01 is
        Port (A : in STD_LOGIC; B : in STD_LOGIC; S : in STD_LOGIC; Y : out STD_LOGIC);
    end component;

    signal mux_low  : STD_LOGIC;
    signal mux_high : STD_LOGIC;

begin
    MUX_L: MUX_4T01 Port Map (D(3 downto 0), S(1 downto 0), mux_low);
    MUX_H: MUX_4T01 Port Map (D(7 downto 4), S(1 downto 0), mux_high);
    MUX_FINAL: MUX_2T01 Port Map (mux_low, mux_high, S(2), Y);
end Structural;
```

24. 16选1MUX

功能说明

16选1多路选择器：根据4位选择信号S从16个输入中选1个输出。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_16T01 is
    Port (
        D : in  STD_LOGIC_VECTOR(15 downto 0);
        S : in  STD_LOGIC_VECTOR(3  downto 0);
        Y : out STD_LOGIC
    );
end MUX_16T01;

architecture Dataflow of MUX_16T01 is
begin
    with S select
        Y <= D(0)  when "0000",
            D(1)  when "0001",
            D(2)  when "0010",
            D(3)  when "0011",
            D(4)  when "0100",
            D(5)  when "0101",
            D(6)  when "0110",
            D(7)  when "0111",
            D(8)  when "1000",
            D(9)  when "1001",
            D(10) when "1010",
            D(11) when "1011",
            D(12) when "1100",
            D(13) when "1101",
            D(14) when "1110",
            D(15) when "1111",
            '0'   when others;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_16T01_Behavioral is
    Port (
        D : in  STD_LOGIC_VECTOR(15 downto 0);
        S : in  STD_LOGIC_VECTOR(3  downto 0);
        Y : out STD_LOGIC
    );
end MUX_16T01_Behavioral;

architecture Behavioral of MUX_16T01_Behavioral is
begin
    process(D, S)
    begin
        case S is
            when "0000" => Y <= D(0);
            when "0001" => Y <= D(1);
            when "0010" => Y <= D(2);
            when "0011" => Y <= D(3);
            when "0100" => Y <= D(4);
            when "0101" => Y <= D(5);
            when "0110" => Y <= D(6);
            when "0111" => Y <= D(7);
            when "1000" => Y <= D(8);
            when "1001" => Y <= D(9);
            when "1010" => Y <= D(10);
            when "1011" => Y <= D(11);
            when "1100" => Y <= D(12);
            when "1101" => Y <= D(13);
            when "1110" => Y <= D(14);
            when "1111" => Y <= D(15);
            when others => Y <= '0';
        end case;
    end process;
end Behavioral;
```

25. 1-2 DEMUX

功能说明

1-2解复用器：根据选择信号S将输入D分配到Y0或Y1。

真值表

S	Y0	Y1
0	D	0
1	0	D

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DEMUX_1T02 is
    Port (
        D : in  STD_LOGIC;
        S : in  STD_LOGIC;
        Y0 : out STD_LOGIC;
        Y1 : out STD_LOGIC
    );
end DEMUX_1T02;

architecture Dataflow of DEMUX_1T02 is
begin
    Y0 <= D when S = '0' else '0';
    Y1 <= D when S = '1' else '0';
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DEMUX_1T02_Behavioral is
    Port (
        D : in  STD_LOGIC;
        S : in  STD_LOGIC;
        Y0 : out STD_LOGIC;
        Y1 : out STD_LOGIC
    );
end DEMUX_1T02_Behavioral;

architecture Behavioral of DEMUX_1T02_Behavioral is
begin
    process(D, S)
    begin
        Y0 <= '0';
        Y1 <= '0';
        case S is
            when '0'    => Y0 <= D;
            when '1'    => Y1 <= D;
            when others => null;
        end case;
    end process;
end Behavioral;
```

26. 1-4 DEMUX

功能说明

1-4解复用器：根据2位选择信号S将输入D分配到Y0~Y3之一。

真值表

S1	S0	Y0	Y1	Y2	Y3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DEMUX_1T04 is
    Port (
        D : in  STD_LOGIC;
        S : in  STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end DEMUX_1T04;

architecture Dataflow of DEMUX_1T04 is
begin
    Y(0) <= D when S = "00" else '0';
    Y(1) <= D when S = "01" else '0';
    Y(2) <= D when S = "10" else '0';
    Y(3) <= D when S = "11" else '0';
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DEMUX_1T04_Behavioral is
    Port (
        D : in  STD_LOGIC;
        S : in  STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end DEMUX_1T04_Behavioral;

architecture Behavioral of DEMUX_1T04_Behavioral is
begin
    process(D, S)
    begin
        Y <= (others => '0');
        case S is
            when "00"  => Y(0) <= D;
            when "01"  => Y(1) <= D;
            when "10"  => Y(2) <= D;
            when "11"  => Y(3) <= D;
            when others => null;
        end case;
    end process;
end Behavioral;
```

27. 1-8 DEMUX

功能说明

1-8解复用器：根据3位选择信号S将输入D分配到Y0~Y7之一。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DEMUX_1T08 is
    Port (
        D : in  STD_LOGIC;
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end DEMUX_1T08;

architecture Dataflow of DEMUX_1T08 is
begin
    Y <= (D => S, others => '0') when false else
        ("0000000" & D) when S = "000" else
        ("000000" & D & '0') when S = "001" else
        ("00000" & D & "00") when S = "010" else
        ("0000" & D & "000") when S = "011" else
        ("000" & D & "0000") when S = "100" else
        ("00" & D & "00000") when S = "101" else
        ('0' & D & "000000") when S = "110" else
        (D & "0000000");
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DEMUX_1T08_Behavioral is
    Port (
        D : in  STD_LOGIC;
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end DEMUX_1T08_Behavioral;

architecture Behavioral of DEMUX_1T08_Behavioral is
begin
    process(D, S)
    begin
        Y <= (others => '0');
        case S is
            when "000" => Y(0) <= D;
            when "001" => Y(1) <= D;
            when "010" => Y(2) <= D;
            when "011" => Y(3) <= D;
            when "100" => Y(4) <= D;
            when "101" => Y(5) <= D;
            when "110" => Y(6) <= D;
            when "111" => Y(7) <= D;
            when others => null;
        end case;
    end process;
end Behavioral;
```

行为描述完整代码 (with-select)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DEMUX_1T08_WithSelect is
    Port (
        D : in  STD_LOGIC;
        S : in  STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end DEMUX_1T08_WithSelect;

architecture Behavioral of DEMUX_1T08_WithSelect is
    signal temp : STD_LOGIC_VECTOR(7 downto 0);
begin
    with S select
        temp <= "00000001" when "000",
               "00000010" when "001",
               "00000100" when "010",
               "00001000" when "011",
               "00010000" when "100",
               "00100000" when "101",
               "01000000" when "110",
               "10000000" when "111",
               "00000000" when others;

    Y <= temp when D = '1' else (others => '0');
end Behavioral;
```

28. 2-4译码器

功能说明

2-4译码器将2位二进制输入译码为4位独热码输出。使能信号E低有效，输出低有效可选。

真值表（输出高有效）

E	A1	A0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_2T04 is
    Port (
        A : in  STD_LOGIC_VECTOR(1 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end DECODER_2T04;

architecture Dataflow of DECODER_2T04 is
begin
    Y <= "0001" when (E = '1' and A = "00") else
        "0010" when (E = '1' and A = "01") else
        "0100" when (E = '1' and A = "10") else
        "1000" when (E = '1' and A = "11") else
        "0000";
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_2T04_Behavioral is
    Port (
        A : in  STD_LOGIC_VECTOR(1 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end DECODER_2T04_Behavioral;

architecture Behavioral of DECODER_2T04_Behavioral is
begin
    process(A, E)
    begin
        if (E = '0') then
            Y <= "0000";
        else
            case A is
                when "00" => Y <= "0001";
                when "01" => Y <= "0010";
                when "10" => Y <= "0100";
                when "11" => Y <= "1000";
                when others => Y <= "0000";
            end case;
        end if;
    end process;
end Behavioral;
```

29. 3-8译码器

功能说明

3-8译码器将3位二进制输入译码为8位独热码输出。

真值表

E	A2	A1	A0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity DECODER_3T08 is
    Port (
        A : in  STD_LOGIC_VECTOR(2 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end DECODER_3T08;

architecture Dataflow of DECODER_3T08 is
begin
    process(A, E)
    begin
        if (E = '0') then
            Y <= (others => '0');
        else
            Y <= (others => '0');
            Y(to_integer(unsigned(A))) <= '1';
        end if;
    end process;
end Dataflow;
```


行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_3T08_Behavioral is
    Port (
        A : in  STD_LOGIC_VECTOR(2 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end DECODER_3T08_Behavioral;

architecture Behavioral of DECODER_3T08_Behavioral is
begin
    process(A, E)
    begin
        if (E = '0') then
            Y <= "00000000";
        else
            case A is
                when "000" => Y <= "00000001";
                when "001" => Y <= "00000010";
                when "010" => Y <= "00000100";
                when "011" => Y <= "00001000";
                when "100" => Y <= "00010000";
                when "101" => Y <= "00100000";
                when "110" => Y <= "01000000";
                when "111" => Y <= "10000000";
                when others => Y <= "00000000";
            end case;
        end if;
    end process;
end Behavioral;
```

行为描述完整代码（when-else）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_3T08_WhenElse is
    Port (
        A : in  STD_LOGIC_VECTOR(2 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end DECODER_3T08_WhenElse;

architecture Behavioral of DECODER_3T08_WhenElse is
begin
    Y <= "00000001" when (E = '1' and A = "000") else
        "00000010" when (E = '1' and A = "001") else
        "00000100" when (E = '1' and A = "010") else
        "00001000" when (E = '1' and A = "011") else
        "00010000" when (E = '1' and A = "100") else
        "00100000" when (E = '1' and A = "101") else
        "01000000" when (E = '1' and A = "110") else
        "10000000" when (E = '1' and A = "111") else
        "00000000";
end Behavioral;
```

30. 4-16译码器

功能说明

4-16译码器将4位二进制输入译码为16位独热码输出。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity DECODER_4TO16 is
    Port (
        A : in  STD_LOGIC_VECTOR(3 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(15 downto 0)
    );
end DECODER_4TO16;

architecture Dataflow of DECODER_4TO16 is
begin
    process(A, E)
    begin
        if (E = '0') then
            Y <= (others => '0');
        else
            Y <= (others => '0');
            Y(to_integer(unsigned(A))) <= '1';
        end if;
    end process;
end Dataflow;
```

行为描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_4TO16_Behavioral is
    Port (
        A : in  STD_LOGIC_VECTOR(3 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(15 downto 0)
    );
end DECODER_4TO16_Behavioral;

architecture Behavioral of DECODER_4TO16_Behavioral is
begin
    process(A, E)
    begin
        if (E = '0') then
            Y <= (others => '0');
        else
            case A is
                when "0000" => Y <= x"0001";
                when "0001" => Y <= x"0002";
                when "0010" => Y <= x"0004";
                when "0011" => Y <= x"0008";
                when "0100" => Y <= x"0010";
                when "0101" => Y <= x"0020";
                when "0110" => Y <= x"0040";
                when "0111" => Y <= x"0080";
                when "1000" => Y <= x"0100";
                when "1001" => Y <= x"0200";
                when "1010" => Y <= x"0400";
                when "1011" => Y <= x"0800";
                when "1100" => Y <= x"1000";
                when "1101" => Y <= x"2000";
                when "1110" => Y <= x"4000";
                when "1111" => Y <= x"8000";
                when others => Y <= (others => '0');
            end case;
        end if;
    end process;
end Behavioral;

```

```
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_4T016_Structural is
    Port (
        A : in  STD_LOGIC_VECTOR(3 downto 0);
        E : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(15 downto 0)
    );
end DECODER_4T016_Structural;

architecture Structural of DECODER_4T016_Structural is

    component DECODER_3T08 is
        Port (A : in STD_LOGIC_VECTOR(2 downto 0); E : in STD_LOGIC; Y : out STD_LOGIC_VECTOR(7 downto 0))
    end component;

    signal en_low  : STD_LOGIC;
    signal en_high : STD_LOGIC;

begin
    en_low  <= E and (not A(3));
    en_high <= E and A(3);

    DEC_LOW: DECODER_3T08
        Port Map (A(2 downto 0), en_low, Y(7 downto 0));

    DEC_HIGH: DECODER_3T08
        Port Map (A(2 downto 0), en_high, Y(15 downto 8));
end Structural;
```

31. 74138风格译码器

功能说明

74138风格3-8译码器：3个使能输入（E1低有效, E2低有效, E3高有效），输出低有效。

真值表

E3	E2	E1	A2	A1	A0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
1	0	0	1	1	1	0	1	1	1	1	1	1	1

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_74138 is
    Port (
        A : in  STD_LOGIC_VECTOR(2 downto 0);
        E1 : in  STD_LOGIC;      -- 低有效使能
        E2 : in  STD_LOGIC;      -- 低有效使能
        E3 : in  STD_LOGIC;      -- 高有效使能
        Y : out STD_LOGIC_VECTOR(7 downto 0) -- 低有效输出
    );
end DECODER_74138;

architecture Dataflow of DECODER_74138 is
    signal enable : STD_LOGIC;
begin
    enable <= E3 and (not E1) and (not E2);

    Y <= "11111110" when (enable = '1' and A = "000") else
        "11111101" when (enable = '1' and A = "001") else
        "11111011" when (enable = '1' and A = "010") else
        "11110111" when (enable = '1' and A = "011") else
        "11101111" when (enable = '1' and A = "100") else
        "11011111" when (enable = '1' and A = "101") else
        "10111111" when (enable = '1' and A = "110") else
        "01111111" when (enable = '1' and A = "111") else
        "11111111";
end Dataflow;
```


行为描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECODER_74138_Behavioral is
    Port (
        A  : in  STD_LOGIC_VECTOR(2 downto 0);
        E1 : in  STD_LOGIC;
        E2 : in  STD_LOGIC;
        E3 : in  STD_LOGIC;
        Y  : out STD_LOGIC_VECTOR(7 downto 0)
    );
end DECODER_74138_Behavioral;

architecture Behavioral of DECODER_74138_Behavioral is
begin
    process(A, E1, E2, E3)
        variable enable : STD_LOGIC;
    begin
        enable := E3 and (not E1) and (not E2);

        if (enable = '0') then
            Y <= "11111111";
        else
            case A is
                when "000" => Y <= "11111110";
                when "001" => Y <= "11111101";
                when "010" => Y <= "11111011";
                when "011" => Y <= "11110111";
                when "100" => Y <= "11101111";
                when "101" => Y <= "11011111";
                when "110" => Y <= "10111111";
                when "111" => Y <= "01111111";
                when others => Y <= "11111111";
            end case;
        end if;
    end process;
end Behavioral;

```

32. 普通编码器

功能说明

普通编码器：4线输入（独热码）→ 2线二进制输出。任一时刻仅允许一个输入有效。

真值表

D3	D2	D1	D0	Y1	Y0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ENCODER_4T02 is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        Y : out STD_LOGIC_VECTOR(1 downto 0)
    );
end ENCODER_4T02;

architecture Dataflow of ENCODER_4T02 is
begin
    Y <= "00" when D = "0001" else
        "01" when D = "0010" else
        "10" when D = "0100" else
        "11" when D = "1000" else
        "00";
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ENCODER_4TO2_Behavioral is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        Y : out STD_LOGIC_VECTOR(1 downto 0)
    );
end ENCODER_4TO2_Behavioral;

architecture Behavioral of ENCODER_4TO2_Behavioral is
begin
    process(D)
    begin
        case D is
            when "0001" => Y <= "00";
            when "0010" => Y <= "01";
            when "0100" => Y <= "10";
            when "1000" => Y <= "11";
            when others => Y <= "00";
        end case;
    end process;
end Behavioral;
```

33. 优先编码器

功能说明

优先编码器：多个输入同时有效时，输出最高优先级的编码（D3最高）。

真值表

D3	D2	D1	D0	Y1	Y0	V
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PRIORITY_ENCODER is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        Y : out STD_LOGIC_VECTOR(1 downto 0);
        V : out STD_LOGIC
    );
end PRIORITY_ENCODER;

architecture Dataflow of PRIORITY_ENCODER is
begin
    Y <= "11" when D(3) = '1' else
        "10" when D(2) = '1' else
        "01" when D(1) = '1' else
        "00" when D(0) = '1' else
        "00";
    V <= D(3) or D(2) or D(1) or D(0);
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PRIORITY_ENCODER_Behavioral is
    Port (
        D : in  STD_LOGIC_VECTOR(3 downto 0);
        Y : out STD_LOGIC_VECTOR(1 downto 0);
        V : out STD_LOGIC
    );
end PRIORITY_ENCODER_Behavioral;

architecture Behavioral of PRIORITY_ENCODER_Behavioral is
begin
    process(D)
    begin
        V <= '0';
        Y <= "00";

        if (D(3) = '1') then
            Y <= "11"; V <= '1';
        elsif (D(2) = '1') then
            Y <= "10"; V <= '1';
        elsif (D(1) = '1') then
            Y <= "01"; V <= '1';
        elsif (D(0) = '1') then
            Y <= "00"; V <= '1';
        else
            Y <= "00"; V <= '0';
        end if;
    end process;
end Behavioral;
```

34. 74148优先编码器

功能说明

74148风格8-3优先编码器：输入/输出均低有效，D7最高优先级，含使能输入EI和使能输出EO、级联输出GS。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ENCODER_74148 is
    Port (
        EI : in  STD_LOGIC;           -- 使能输入（低有效）
        D  : in  STD_LOGIC_VECTOR(7 downto 0); -- 数据输入（低有效）
        Y  : out STD_LOGIC_VECTOR(2 downto 0); -- 编码输出（低有效）
        GS : out STD_LOGIC;           -- 组选择输出（低有效）
        EO : out STD_LOGIC           -- 使能输出（低有效）
    );
end ENCODER_74148;

architecture Dataflow of ENCODER_74148 is
begin
    process(EI, D)
    begin
        if (EI = '1') then
            Y <= "111";
            GS <= '1';
            EO <= '1';
        elsif (D(7) = '0') then
            Y <= "000"; GS <= '0'; EO <= '1';
        elsif (D(6) = '0') then
            Y <= "001"; GS <= '0'; EO <= '1';
        elsif (D(5) = '0') then
            Y <= "010"; GS <= '0'; EO <= '1';
        elsif (D(4) = '0') then
            Y <= "011"; GS <= '0'; EO <= '1';
        elsif (D(3) = '0') then
            Y <= "100"; GS <= '0'; EO <= '1';
        elsif (D(2) = '0') then
            Y <= "101"; GS <= '0'; EO <= '1';
        elsif (D(1) = '0') then
            Y <= "110"; GS <= '0'; EO <= '1';
        elsif (D(0) = '0') then
            Y <= "111"; GS <= '0'; EO <= '1';
        else
            Y <= "111";
        end if;
    end process;
end Dataflow of ENCODER_74148;

```

```
        GS <= '1';  
        EO <= '0';  
    end if;  
end process;  
end Dataflow;
```

行为描述完整代码 (if-elsif 优先级链)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ENCODER_74148_Behavioral is
    Port (
        EI : in  STD_LOGIC;
        D  : in  STD_LOGIC_VECTOR(7 downto 0);
        Y   : out STD_LOGIC_VECTOR(2 downto 0);
        GS  : out STD_LOGIC;
        EO  : out STD_LOGIC
    );
end ENCODER_74148_Behavioral;

architecture Behavioral of ENCODER_74148_Behavioral is
begin
    process(EI, D)
    begin
        if (EI = '1') then
            Y <= "111"; GS <= '1'; EO <= '1';
        else
            if (D = "11111111") then
                Y <= "111"; GS <= '1'; EO <= '0';
            elsif (D(7) = '0') then
                Y <= "000"; GS <= '0'; EO <= '1';
            elsif (D(6) = '0') then
                Y <= "001"; GS <= '0'; EO <= '1';
            elsif (D(5) = '0') then
                Y <= "010"; GS <= '0'; EO <= '1';
            elsif (D(4) = '0') then
                Y <= "011"; GS <= '0'; EO <= '1';
            elsif (D(3) = '0') then
                Y <= "100"; GS <= '0'; EO <= '1';
            elsif (D(2) = '0') then
                Y <= "101"; GS <= '0'; EO <= '1';
            elsif (D(1) = '0') then
                Y <= "110"; GS <= '0'; EO <= '1';
            elsif (D(0) = '0') then
                Y <= "111"; GS <= '0'; EO <= '1';
            else

```

```
        Y <= "111"; GS <= '1'; EO <= '1';
    end if;
end if;
end process;
end Behavioral;
```

35. 奇校验器

功能说明

奇校验器：检测输入中'1'的个数是否为奇数。奇数个'1'时输出'1'。

真值表（4位输入）

A3	A2	A1	A0	P_odd
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ODD_PARITY is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        P : out STD_LOGIC
    );
end ODD_PARITY;

architecture Dataflow of ODD_PARITY is
begin
    P <= D(0) xor D(1) xor D(2) xor D(3) xor D(4) xor D(5) xor D(6) xor D(7);
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ODD_PARITY_Behavioral is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        P : out STD_LOGIC
    );
end ODD_PARITY_Behavioral;

architecture Behavioral of ODD_PARITY_Behavioral is
begin
    process(D)
        variable ones_count : INTEGER range 0 to 8;
    begin
        ones_count := 0;
        for i in 0 to 7 loop
            if (D(i) = '1') then
                ones_count := ones_count + 1;
            end if;
        end loop;

        if (ones_count mod 2 = 1) then
            P <= '1';
        else
            P <= '0';
        end if;
    end process;
end Behavioral;
```


结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ODD_PARITY_Structural is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        P : out STD_LOGIC
    );
end ODD_PARITY_Structural;

architecture Structural of ODD_PARITY_Structural is
    signal xor_01  : STD_LOGIC;
    signal xor_23  : STD_LOGIC;
    signal xor_45  : STD_LOGIC;
    signal xor_67  : STD_LOGIC;
    signal xor_0123 : STD_LOGIC;
    signal xor_4567 : STD_LOGIC;
begin
    xor_01  <= D(0) xor D(1);
    xor_23  <= D(2) xor D(3);
    xor_45  <= D(4) xor D(5);
    xor_67  <= D(6) xor D(7);
    xor_0123 <= xor_01 xor xor_23;
    xor_4567 <= xor_45 xor xor_67;
    P <= xor_0123 xor xor_4567;
end Structural;
```

36. 偶校验器

功能说明

偶校验器：检测输入中'1'的个数是否为偶数。偶数个'1'时输出'1'。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity EVEN_PARITY is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        P : out STD_LOGIC
    );
end EVEN_PARITY;

architecture Dataflow of EVEN_PARITY is
begin
    P <= not (D(0) xor D(1) xor D(2) xor D(3) xor D(4) xor D(5) xor D(6) xor D(7));
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity EVEN_PARITY_Behavioral is
    Port (
        D : in  STD_LOGIC_VECTOR(7 downto 0);
        P : out STD_LOGIC
    );
end EVEN_PARITY_Behavioral;

architecture Behavioral of EVEN_PARITY_Behavioral is
begin
    process(D)
        variable ones_count : INTEGER range 0 to 8;
    begin
        ones_count := 0;
        for i in 0 to 7 loop
            if (D(i) = '1') then
                ones_count := ones_count + 1;
            end if;
        end loop;

        if (ones_count mod 2 = 0) then
            P <= '1';
        else
            P <= '0';
        end if;
    end process;
end Behavioral;
```

37. 校验发生器

功能说明

奇校验发生器：在7位数据后附加1位奇校验位，使8位中'1'的总数为奇数。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PARITY_GENERATOR_ODD is
    Port (
        DATA_IN  : in  STD_LOGIC_VECTOR(6 downto 0);
        DATA_OUT  : out STD_LOGIC_VECTOR(7 downto 0)
    );
end PARITY_GENERATOR_ODD;

architecture Dataflow of PARITY_GENERATOR_ODD is
    signal parity_bit : STD_LOGIC;
begin
    parity_bit <= not (DATA_IN(0) xor DATA_IN(1) xor DATA_IN(2) xor DATA_IN(3)
                      xor DATA_IN(4) xor DATA_IN(5) xor DATA_IN(6));
    DATA_OUT <= DATA_IN & parity_bit;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PARITY_GENERATOR_EVEN is
    Port (
        DATA_IN  : in  STD_LOGIC_VECTOR(6 downto 0);
        DATA_OUT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end PARITY_GENERATOR_EVEN;

architecture Behavioral of PARITY_GENERATOR_EVEN is
begin
    process(DATA_IN)
        variable ones_count : INTEGER range 0 to 7;
        variable parity_bit : STD_LOGIC;
    begin
        ones_count := 0;
        for i in 0 to 6 loop
            if (DATA_IN(i) = '1') then
                ones_count := ones_count + 1;
            end if;
        end loop;

        if (ones_count mod 2 = 1) then
            parity_bit := '1';
        else
            parity_bit := '0';
        end if;

        DATA_OUT <= DATA_IN & parity_bit;
    end process;
end Behavioral;
```

38. 校验检测器

功能说明

奇校验检测器：接收8位数据（含校验位），检测'1'的总数是否为奇数，输出错误标志。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PARITY_CHECKER_ODD is
    Port (
        DATA_IN : in  STD_LOGIC_VECTOR(7 downto 0);
        ERROR     : out STD_LOGIC
    );
end PARITY_CHECKER_ODD;

architecture Dataflow of PARITY_CHECKER_ODD is
begin
    ERROR <= not (DATA_IN(0) xor DATA_IN(1) xor DATA_IN(2) xor DATA_IN(3)
                 xor DATA_IN(4) xor DATA_IN(5) xor DATA_IN(6) xor DATA_IN(7));
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PARITY_CHECKER_EVEN is
    Port (
        DATA_IN : in  STD_LOGIC_VECTOR(7 downto 0);
        ERROR     : out STD_LOGIC
    );
end PARITY_CHECKER_EVEN;

architecture Behavioral of PARITY_CHECKER_EVEN is
begin
    process(DATA_IN)
        variable ones_count : INTEGER range 0 to 8;
    begin
        ones_count := 0;
        for i in 0 to 7 loop
            if (DATA_IN(i) = '1') then
                ones_count := ones_count + 1;
            end if;
        end loop;

        if (ones_count mod 2 = 0) then
            ERROR <= '0';
        else
            ERROR <= '1';
        end if;
    end process;
end Behavioral;
```

39. 二进制转格雷码

功能说明

将N位二进制码转换为N位格雷码。公式： $G(i) = B(i) \text{ XOR } B(i+1)$ ，最高位 $G(N-1) = B(N-1)$ 。

真值表（4位）

B3	B2	B1	B0	G3	G2	G1	G0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BIN_TO_GRAY is
    Port (
        BIN   : in  STD_LOGIC_VECTOR(3 downto 0);
        GRAY  : out STD_LOGIC_VECTOR(3 downto 0)
    );
end BIN_TO_GRAY;

architecture Dataflow of BIN_TO_GRAY is
begin
    GRAY(3) <= BIN(3);
    GRAY(2) <= BIN(3) xor BIN(2);
    GRAY(1) <= BIN(2) xor BIN(1);
    GRAY(0) <= BIN(1) xor BIN(0);
end Dataflow;
```

行为描述完整代码（8位通用版本）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BIN_TO_GRAY_8BIT is
    Port (
        BIN   : in  STD_LOGIC_VECTOR(7 downto 0);
        GRAY  : out STD_LOGIC_VECTOR(7 downto 0)
    );
end BIN_TO_GRAY_8BIT;

architecture Behavioral of BIN_TO_GRAY_8BIT is
begin
    process(BIN)
    begin
        GRAY(7) <= BIN(7);
        for i in 6 downto 0 loop
            GRAY(i) <= BIN(i+1) xor BIN(i);
        end loop;
    end process;
end Behavioral;
```

行为描述完整代码（4位 case方式）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BIN_TO_GRAY_Case is
    Port (
        BIN   : in  STD_LOGIC_VECTOR(3 downto 0);
        GRAY  : out STD_LOGIC_VECTOR(3 downto 0)
    );
end BIN_TO_GRAY_Case;

architecture Behavioral of BIN_TO_GRAY_Case is
begin
    process(BIN)
    begin
        case BIN is
            when "0000" => GRAY <= "0000";
            when "0001" => GRAY <= "0001";
            when "0010" => GRAY <= "0011";
            when "0011" => GRAY <= "0010";
            when "0100" => GRAY <= "0110";
            when "0101" => GRAY <= "0111";
            when "0110" => GRAY <= "0101";
            when "0111" => GRAY <= "0100";
            when "1000" => GRAY <= "1100";
            when "1001" => GRAY <= "1101";
            when "1010" => GRAY <= "1111";
            when "1011" => GRAY <= "1110";
            when "1100" => GRAY <= "1010";
            when "1101" => GRAY <= "1011";
            when "1110" => GRAY <= "1001";
            when "1111" => GRAY <= "1000";
            when others => GRAY <= "0000";
        end case;
    end process;
end Behavioral;
```

40. 格雷码转二进制

功能说明

将N位格雷码转换为N位二进制码。公式： $B(N-1) = G(N-1)$ ， $B(i) = G(i) \text{ XOR } B(i+1)$ 。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity GRAY_TO_BIN is
    Port (
        GRAY : in  STD_LOGIC_VECTOR(3 downto 0);
        BIN  : out STD_LOGIC_VECTOR(3 downto 0)
    );
end GRAY_TO_BIN;

architecture Dataflow of GRAY_TO_BIN is
begin
    BIN(3) <= GRAY(3);
    BIN(2) <= GRAY(3) xor GRAY(2);
    BIN(1) <= GRAY(3) xor GRAY(2) xor GRAY(1);
    BIN(0) <= GRAY(3) xor GRAY(2) xor GRAY(1) xor GRAY(0);
end Dataflow;
```

行为描述完整代码（通用N位）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity GRAY_TO_BIN_8BIT is
    Port (
        GRAY : in  STD_LOGIC_VECTOR(7 downto 0);
        BIN  : out STD_LOGIC_VECTOR(7 downto 0)
    );
end GRAY_TO_BIN_8BIT;

architecture Behavioral of GRAY_TO_BIN_8BIT is
begin
    process(GRAY)
        variable temp : STD_LOGIC_VECTOR(7 downto 0);
    begin
        temp(7) := GRAY(7);
        for i in 6 downto 0 loop
            temp(i) := temp(i+1) xor GRAY(i);
        end loop;
        BIN <= temp;
    end process;
end Behavioral;
```

行为描述完整代码（4位 case方式）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity GRAY_TO_BIN_Case is
    Port (
        GRAY : in  STD_LOGIC_VECTOR(3 downto 0);
        BIN   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end GRAY_TO_BIN_Case;

architecture Behavioral of GRAY_TO_BIN_Case is
begin
    process(GRAY)
    begin
        case GRAY is
            when "0000" => BIN <= "0000";
            when "0001" => BIN <= "0001";
            when "0011" => BIN <= "0010";
            when "0010" => BIN <= "0011";
            when "0110" => BIN <= "0100";
            when "0111" => BIN <= "0101";
            when "0101" => BIN <= "0110";
            when "0100" => BIN <= "0111";
            when "1100" => BIN <= "1000";
            when "1101" => BIN <= "1001";
            when "1111" => BIN <= "1010";
            when "1110" => BIN <= "1011";
            when "1010" => BIN <= "1100";
            when "1011" => BIN <= "1101";
            when "1001" => BIN <= "1110";
            when "1000" => BIN <= "1111";
            when others => BIN <= "0000";
        end case;
    end process;
end Behavioral;
```

41. BCD转二进制

功能说明

将2位BCD码（8位，00~99）转换为7位二进制数。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BCD_TO_BIN is
    Port (
        BCD : in  STD_LOGIC_VECTOR(7 downto 0);    -- 2位BCD（高4位十位，低4位个位）
        BIN : out STD_LOGIC_VECTOR(6 downto 0)      -- 7位二进制（0~99）
    );
end BCD_TO_BIN;

architecture Dataflow of BCD_TO_BIN is
    signal tens      : STD_LOGIC_VECTOR(3 downto 0);
    signal ones      : STD_LOGIC_VECTOR(3 downto 0);
    signal tens_x10   : STD_LOGIC_VECTOR(6 downto 0);
begin
    tens <= BCD(7 downto 4);
    ones <= BCD(3 downto 0);

    tens_x10 <= (tens & "000") + ("00" & tens & '0');
    BIN <= tens_x10 + ("000" & ones);
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity BCD_TO_BIN_Behavioral is
    Port (
        BCD : in  STD_LOGIC_VECTOR(7 downto 0);
        BIN  : out STD_LOGIC_VECTOR(6 downto 0)
    );
end BCD_TO_BIN_Behavioral;

architecture Behavioral of BCD_TO_BIN_Behavioral is
begin
    process(BCD)
        variable tens : INTEGER range 0 to 9;
        variable ones  : INTEGER range 0 to 9;
        variable result : INTEGER range 0 to 99;
    begin
        tens := to_integer(unsigned(BCD(7 downto 4)));
        ones  := to_integer(unsigned(BCD(3 downto 0)));
        result := tens * 10 + ones;
        BIN <= std_logic_vector(to_unsigned(result, 7));
    end process;
end Behavioral;
```

42. 二进制转BCD

功能说明

将8位二进制数(0~255)转换为3位BCD码（12位，百位+十位+个位）。采用"左移加3"算法。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity BIN_TO_BCD is
    Port (
        BIN : in  STD_LOGIC_VECTOR(7 downto 0);
        BCD : out STD_LOGIC_VECTOR(11 downto 0)
    );
end BIN_TO_BCD;

architecture Dataflow of BIN_TO_BCD is
begin
    process(BIN)
        variable temp : unsigned(19 downto 0);
    begin
        temp := (others => '0');
        temp(7 downto 0) := unsigned(BIN);

        for i in 0 to 7 loop
            if (temp(11 downto 8) > 4) then
                temp(11 downto 8) := temp(11 downto 8) + 3;
            end if;
            if (temp(15 downto 12) > 4) then
                temp(15 downto 12) := temp(15 downto 12) + 3;
            end if;
            if (temp(19 downto 16) > 4) then
                temp(19 downto 16) := temp(19 downto 16) + 3;
            end if;
            temp := temp(18 downto 0) & '0';
        end loop;

        BCD <= std_logic_vector(temp(19 downto 8));
    end process;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity BIN_TO_BCD_Behavioral is
    Port (
        BIN : in  STD_LOGIC_VECTOR(7 downto 0);
        BCD : out STD_LOGIC_VECTOR(11 downto 0)
    );
end BIN_TO_BCD_Behavioral;

architecture Behavioral of BIN_TO_BCD_Behavioral is
begin
    process(BIN)
        variable bin_val : INTEGER range 0 to 255;
        variable hundreds : INTEGER range 0 to 2;
        variable tens      : INTEGER range 0 to 9;
        variable ones      : INTEGER range 0 to 9;
    begin
        bin_val := to_integer(unsigned(BIN));

        hundreds := bin_val / 100;
        tens      := (bin_val mod 100) / 10;
        ones      := bin_val mod 10;

        BCD(11 downto 8) <= std_logic_vector(to_unsigned(hundreds, 4));
        BCD(7 downto 4)  <= std_logic_vector(to_unsigned(tens, 4));
        BCD(3 downto 0)  <= std_logic_vector(to_unsigned(ones, 4));
    end process;
end Behavioral;
```

43. 七段译码器

功能说明

七段数码管译码器：将4位BCD码译码为七段数码管段选信号（共阴极，段选高有效）。段排序：g f e d c b a。

真值表

BCD	显示	a	b	c	d	e	f	g
0000	0	1	1	1	1	1	1	0
0001	1	0	1	1	0	0	0	0
0010	2	1	1	0	1	1	0	1
0011	3	1	1	1	1	0	0	1
0100	4	0	1	1	0	0	1	1
0101	5	1	0	1	1	0	1	1
0110	6	1	0	1	1	1	1	1
0111	7	1	1	1	0	0	0	0
1000	8	1	1	1	1	1	1	1
1001	9	1	1	1	1	0	1	1
1010	A	1	1	1	0	1	1	1
1011	B	0	0	1	1	1	1	1
1100	C	1	0	0	1	1	1	0
1101	D	0	1	1	1	1	0	1
1110	E	1	0	0	1	1	1	1

BCD	显示	a	b	c	d	e	f	g
1111	F	1	0	0	0	1	1	1

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SEVEN_SEG_DECODER is
    Port (
        BCD : in  STD_LOGIC_VECTOR(3 downto 0);
        SEG : out STD_LOGIC_VECTOR(6 downto 0)    -- {g,f,e,d,c,b,a}
    );
end SEVEN_SEG_DECODER;

architecture Dataflow of SEVEN_SEG_DECODER is
begin
    with BCD select
        SEG <= "0111111" when "0000",    -- 0
               "0000110" when "0001",    -- 1
               "1011011" when "0010",    -- 2
               "1001111" when "0011",    -- 3
               "1100110" when "0100",    -- 4
               "1101101" when "0101",    -- 5
               "1111101" when "0110",    -- 6
               "0000111" when "0111",    -- 7
               "1111111" when "1000",    -- 8
               "1101111" when "1001",    -- 9
               "1110111" when "1010",    -- A
               "1111100" when "1011",    -- B
               "0111001" when "1100",    -- C
               "1011110" when "1101",    -- D
               "1111001" when "1110",    -- E
               "1110001" when "1111",    -- F
               "0000000" when others;
end Dataflow;

```

行为描述完整代码（共阴极）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SEVEN_SEG_DECODER_CA is
    Port (
        BCD : in  STD_LOGIC_VECTOR(3 downto 0);
        SEG : out STD_LOGIC_VECTOR(6 downto 0)
    );
end SEVEN_SEG_DECODER_CA;

architecture Behavioral of SEVEN_SEG_DECODER_CA is
begin
    process(BCD)
    begin
        case BCD is
            when "0000" => SEG <= "0111111";
            when "0001" => SEG <= "0000110";
            when "0010" => SEG <= "1011011";
            when "0011" => SEG <= "1001111";
            when "0100" => SEG <= "1100110";
            when "0101" => SEG <= "1101101";
            when "0110" => SEG <= "1111101";
            when "0111" => SEG <= "0000111";
            when "1000" => SEG <= "1111111";
            when "1001" => SEG <= "1101111";
            when "1010" => SEG <= "1110111";
            when "1011" => SEG <= "1111100";
            when "1100" => SEG <= "0111001";
            when "1101" => SEG <= "1011110";
            when "1110" => SEG <= "1111001";
            when "1111" => SEG <= "1110001";
            when others => SEG <= "0000000";
        end case;
    end process;
end Behavioral;
```

行为描述完整代码（共阳极，输出低有效）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SEVEN_SEG_DECODER_CC is
    Port (
        BCD : in  STD_LOGIC_VECTOR(3 downto 0);
        SEG : out STD_LOGIC_VECTOR(6 downto 0)
    );
end SEVEN_SEG_DECODER_CC;

architecture Behavioral of SEVEN_SEG_DECODER_CC is
begin
    process(BCD)
    begin
        case BCD is
            when "0000" => SEG <= "1000000";
            when "0001" => SEG <= "1111001";
            when "0010" => SEG <= "0100100";
            when "0011" => SEG <= "0110000";
            when "0100" => SEG <= "0011001";
            when "0101" => SEG <= "0010010";
            when "0110" => SEG <= "0000010";
            when "0111" => SEG <= "1111000";
            when "1000" => SEG <= "0000000";
            when "1001" => SEG <= "0010000";
            when "1010" => SEG <= "0001000";
            when "1011" => SEG <= "0000011";
            when "1100" => SEG <= "1000110";
            when "1101" => SEG <= "0100001";
            when "1110" => SEG <= "0000110";
            when "1111" => SEG <= "0001110";
            when others => SEG <= "1111111";
        end case;
    end process;
end Behavioral;
```

44. 七段动态扫描驱动

功能说明

4位数码管动态扫描驱动：分时扫描4个数码管，利用视觉暂留效应实现同时显示。

数据流描述完整代码


```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity SEVEN_SEG_SCAN is
    Port (
        CLK      : in  STD_LOGIC;           -- 扫描时钟 (>200Hz)
        RST      : in  STD_LOGIC;           -- 复位
        DATA_IN  : in  STD_LOGIC_VECTOR(15 downto 0); -- 4位BCD数据
        SEG      : out STD_LOGIC_VECTOR(6 downto 0); -- 段选
        AN       : out STD_LOGIC_VECTOR(3 downto 0)  -- 位选 (低有效)
    );
end SEVEN_SEG_SCAN;

architecture Dataflow of SEVEN_SEG_SCAN is
    signal scan_cnt : unsigned(1 downto 0);
    signal div_cnt  : unsigned(16 downto 0);
    signal scan_clk : STD_LOGIC;
    signal current_digit : STD_LOGIC_VECTOR(3 downto 0);
begin

    -- 分频器
    process(CLK, RST)
    begin
        if (RST = '1') then
            div_cnt <= (others => '0');
            scan_clk <= '0';
        elsif rising_edge(CLK) then
            if (div_cnt = 99999) then
                div_cnt <= (others => '0');
                scan_clk <= not scan_clk;
            else
                div_cnt <= div_cnt + 1;
            end if;
        end if;
    end process;

    -- 扫描计数器
    process(scan_clk, RST)

```

```

begin
    if (RST = '1') then
        scan_cnt <= (others => '0');
    elsif rising_edge(scan_clk) then
        scan_cnt <= scan_cnt + 1;
    end if;
end process;

-- 位选译码
with scan_cnt select
    AN <= "1110" when "00",
         "1101" when "01",
         "1011" when "10",
         "0111" when "11",
         "1111" when others;

-- 数据选择
with scan_cnt select
    current_digit <= DATA_IN(3 downto 0)  when "00",
                    DATA_IN(7 downto 4)  when "01",
                    DATA_IN(11 downto 8)  when "10",
                    DATA_IN(15 downto 12) when "11",
                    "0000"                  when others;

-- 段选译码
with current_digit select
    SEG <= "0111111" when "0000",
          "0000110"  when "0001",
          "1011011"  when "0010",
          "1001111"  when "0011",
          "1100110"  when "0100",
          "1101101"  when "0101",
          "1111101"  when "0110",
          "0000111"  when "0111",
          "1111111"  when "1000",
          "1101111"  when "1001",
          "0000000"  when others;

end Dataflow;

```

行为描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity SEVEN_SEG_SCAN_Behavioral is
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        DATA_IN : in  STD_LOGIC_VECTOR(15 downto 0);
        SEG      : out STD_LOGIC_VECTOR(6 downto 0);
        AN       : out STD_LOGIC_VECTOR(3 downto 0)
    );
end SEVEN_SEG_SCAN_Behavioral;

architecture Behavioral of SEVEN_SEG_SCAN_Behavioral is
    signal scan_counter : INTEGER range 0 to 3;
    signal clk_divider   : INTEGER range 0 to 99999;
    signal scan_tick     : STD_LOGIC;
begin

    -- 时钟分频
    process(CLK, RST)
    begin
        if (RST = '1') then
            clk_divider <= 0;
            scan_tick <= '0';
        elsif rising_edge(CLK) then
            if (clk_divider = 99999) then
                clk_divider <= 0;
                scan_tick <= '1';
            else
                clk_divider <= clk_divider + 1;
                scan_tick <= '0';
            end if;
        end if;
    end process;

    -- 扫描控制
    process(CLK, RST)

```

```

variable digit : STD_LOGIC_VECTOR(3 downto 0);
begin
    if (RST = '1') then
        scan_counter <= 0;
        SEG <= "0000000";
        AN <= "1111";
    elsif rising_edge(CLK) then
        if (scan_tick = '1') then
            case scan_counter is
                when 0 =>
                    AN <= "1110";
                    digit := DATA_IN(3 downto 0);
                    scan_counter <= 1;
                when 1 =>
                    AN <= "1101";
                    digit := DATA_IN(7 downto 4);
                    scan_counter <= 2;
                when 2 =>
                    AN <= "1011";
                    digit := DATA_IN(11 downto 8);
                    scan_counter <= 3;
                when 3 =>
                    AN <= "0111";
                    digit := DATA_IN(15 downto 12);
                    scan_counter <= 0;
                when others =>
                    AN <= "1111";
                    digit := "0000";
                    scan_counter <= 0;
            end case;

            case digit is
                when "0000" => SEG <= "0111111";
                when "0001" => SEG <= "0000110";
                when "0010" => SEG <= "1011011";
                when "0011" => SEG <= "1001111";
                when "0100" => SEG <= "1100110";
                when "0101" => SEG <= "1101101";
                when "0110" => SEG <= "1111101";
            end case;
        end if;
    end if;
end begin;

```

```
        when "0111" => SEG <= "0000111";
        when "1000" => SEG <= "1111111";
        when "1001" => SEG <= "1101111";
        when others => SEG <= "0000000";
    end case;
end if;
end if;
end process;

end Behavioral;
```

45. 简单ALU

功能说明

8位简单ALU支持8种运算：加、减、与、或、异或、非、左移、右移。

真值表

OP(2:0)	运算	结果Y
000	ADD	$A + B + Cin$
001	SUB	$A - B - Cin$
010	AND	$A \text{ AND } B$
011	OR	$A \text{ OR } B$
100	XOR	$A \text{ XOR } B$
101	NOT	$\text{NOT } A$
110	SHL	$A \ll 1$

OP(2:0)	运算	结果Y
111	SHR	$A \gg 1$

数据流描述完整代码


```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity ALU_SIMPLE is
    Port (
        A      : in  STD_LOGIC_VECTOR(7 downto 0);
        B      : in  STD_LOGIC_VECTOR(7 downto 0);
        OP     : in  STD_LOGIC_VECTOR(2 downto 0);
        CIN    : in  STD_LOGIC;
        Y      : out STD_LOGIC_VECTOR(7 downto 0);
        COUT   : out STD_LOGIC;
        ZERO   : out STD_LOGIC;
        SIGN   : out STD_LOGIC
    );
end ALU_SIMPLE;

architecture Dataflow of ALU_SIMPLE is
    signal result      : STD_LOGIC_VECTOR(8 downto 0);
    signal result_7    : STD_LOGIC_VECTOR(7 downto 0);
    signal temp_add    : STD_LOGIC_VECTOR(8 downto 0);
    signal temp_sub    : STD_LOGIC_VECTOR(8 downto 0);
begin

    temp_add <= ('0' & A) + ('0' & B) + CIN;
    temp_sub <= ('0' & A) - ('0' & B) - CIN;

    with OP select
        result <= temp_add                                when "000",
                  temp_sub                                when "001",
                  '0' & (A and B)                        when "010",
                  '0' & (A or B)                         when "011",
                  '0' & (A xor B)                        when "100",
                  '0' & (not A)                          when "101",
                  A(6 downto 0) & '0' & '0'              when "110",
                  '0' & '0' & A(7 downto 1)              when "111",
                  (others => '0')                        when others;

    result_7 <= result(7 downto 0);

```

```
Y    <= result_7;  
COUT <= result(8);  
  
ZERO <= '1' when (result_7 = x"00") else '0';  
SIGN <= result_7(7);
```

```
end Dataflow;
```

行为描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity ALU_SIMPLE_Behavioral is
    Port (
        A    : in  STD_LOGIC_VECTOR(7 downto 0);
        B    : in  STD_LOGIC_VECTOR(7 downto 0);
        OP    : in  STD_LOGIC_VECTOR(2 downto 0);
        Y     : out STD_LOGIC_VECTOR(7 downto 0);
        COUT  : out STD_LOGIC;
        ZERO  : out STD_LOGIC;
        SIGN  : out STD_LOGIC
    );
end ALU_SIMPLE_Behavioral;

architecture Behavioral of ALU_SIMPLE_Behavioral is
begin
    process(A, B, OP)
        variable temp : STD_LOGIC_VECTOR(8 downto 0);
    begin
        COUT <= '0';

        case OP is
            when "000" =>
                temp := ('0' & A) + ('0' & B);
                COUT <= temp(8);
            when "001" =>
                temp := ('0' & A) - ('0' & B);
                COUT <= temp(8);
            when "010" =>
                temp := '0' & (A and B);
            when "011" =>
                temp := '0' & (A or B);
            when "100" =>
                temp := '0' & (A xor B);
            when "101" =>
                temp := '0' & (not A);
            when "110" =>

```

```
        temp := '0' & A(6 downto 0) & '0';
    when "111" =>
        temp := '0' & '0' & A(7 downto 1);
    when others =>
        temp := (others => '0');
    end case;

    Y <= temp(7 downto 0);

    if (temp(7 downto 0) = x"00") then
        ZERO <= '1';
    else
        ZERO <= '0';
    end if;

    SIGN <= temp(7);
end process;
end Behavioral;
```

46. 74181风格ALU

功能说明

74181风格4位ALU：支持16种逻辑运算和16种算术运算，由模式选择M和功能选择S控制。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity ALU_74181 is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        S      : in  STD_LOGIC_VECTOR(3 downto 0);    -- 功能选择
        M      : in  STD_LOGIC;                       -- 模式: 0=算术, 1=逻辑
        CIN     : in  STD_LOGIC;                       -- 进位输入 (低有效)
        Y      : out STD_LOGIC_VECTOR(3 downto 0);
        COUT    : out STD_LOGIC;                       -- 进位输出 (低有效)
        AEB     : out STD_LOGIC;                       -- A=B输出
        X      : out STD_LOGIC;                       -- 进位传播
        Gbar    : out STD_LOGIC;                      -- 进位生成
    );
end ALU_74181;

architecture Dataflow of ALU_74181 is
    signal result : STD_LOGIC_VECTOR(4 downto 0);
begin
    process(A, B, S, M, CIN)
        variable temp : STD_LOGIC_VECTOR(4 downto 0);
    begin
        if (M = '1') then
            -- 逻辑运算模式
            case S is
                when "0000" => temp := '0' & (not A);
                when "0001" => temp := '0' & (not (A or B));
                when "0010" => temp := '0' & ((not A) and B);
                when "0011" => temp := '0' & x"0";
                when "0100" => temp := '0' & (not (A and B));
                when "0101" => temp := '0' & (not B);
                when "0110" => temp := '0' & (A xor B);
                when "0111" => temp := '0' & (A and (not B));
                when "1000" => temp := '0' & ((not A) or B);
                when "1001" => temp := '0' & (not (A xor B));
                when "1010" => temp := '0' & B;
            end case;
        end if;
    end process;
end Dataflow;

```

```

        when "1011" => temp := '0' & (A and B);
        when "1100" => temp := '0' & x"F";
        when "1101" => temp := '0' & (A or (not B));
        when "1110" => temp := '0' & (A or B);
        when "1111" => temp := '0' & A;
        when others => temp := (others => '0');
    end case;
else
    -- 算术运算模式
    case S is
        when "0000" => temp := ('0' & A) + (not CIN);
        when "0001" => temp := ('0' & (A or B)) + (not CIN);
        when "0010" => temp := ('0' & (A or (not B))) + (not CIN);
        when "0011" => temp := ('0' & x"F") + (not CIN);
        when "0100" => temp := ('0' & A) + ('0' & (A and (not B))) + (not CIN);
        when "0101" => temp := ('0' & (A or B)) + ('0' & (A and (not B))) + (not CIN);
        when "0110" => temp := ('0' & A) - ('0' & B) - (not (not CIN));
        when "0111" => temp := ('0' & (A and (not B))) - ('0' & x"1") + (not CIN);
        when "1000" => temp := ('0' & A) + ('0' & (A and B)) + (not CIN);
        when "1001" => temp := ('0' & A) + ('0' & B) + (not CIN);
        when "1010" => temp := ('0' & (A or (not B))) + ('0' & (A and B)) + (not CIN);
        when "1011" => temp := ('0' & (A and B)) - ('0' & x"1") + (not CIN);
        when "1100" => temp := ('0' & A) + ('0' & A) + (not CIN);
        when "1101" => temp := ('0' & (A or B)) + ('0' & A) + (not CIN);
        when "1110" => temp := ('0' & (A or (not B))) + ('0' & A) + (not CIN);
        when "1111" => temp := ('0' & A) - ('0' & x"1") + (not CIN);
        when others => temp := (others => '0');
    end case;
end if;

result <= temp;
end process;

Y    <= result(3 downto 0);
COUT <= not result(4);

AEB <= '1' when (result(3 downto 0) = x"F") else '0';
end Dataflow;

```


行为描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity ALU_74181_Behavioral is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);
        B    : in  STD_LOGIC_VECTOR(3 downto 0);
        S    : in  STD_LOGIC_VECTOR(3 downto 0);
        M    : in  STD_LOGIC;
        CIN  : in  STD_LOGIC;
        Y    : out STD_LOGIC_VECTOR(3 downto 0);
        COUT : out STD_LOGIC;
        AEB  : out STD_LOGIC
    );
end ALU_74181_Behavioral;

architecture Behavioral of ALU_74181_Behavioral is
begin
    process(A, B, S, M, CIN)
        variable temp : STD_LOGIC_VECTOR(4 downto 0);
    begin
        if (M = '1') then
            -- 逻辑运算
            case S is
                when "0000" => temp := '0' & (not A);
                when "0001" => temp := '0' & (not (A or B));
                when "0010" => temp := '0' & ((not A) and B);
                when "0011" => temp := '0' & x"0";
                when "0100" => temp := '0' & (not (A and B));
                when "0101" => temp := '0' & (not B);
                when "0110" => temp := '0' & (A xor B);
                when "0111" => temp := '0' & (A and (not B));
                when "1000" => temp := '0' & ((not A) or B);
                when "1001" => temp := '0' & (not (A xor B));
                when "1010" => temp := '0' & B;
                when "1011" => temp := '0' & (A and B);
                when "1100" => temp := '0' & x"F";
                when "1101" => temp := '0' & (A or (not B));
            end case;
        end if;
    end process;
end Behavioral;

```

```

        when "1110" => temp := '0' & (A or B);
        when "1111" => temp := '0' & A;
        when others => temp := (others => '0');
    end case;
else
    -- 算术运算
    case S is
        when "0000" => temp := ('0' & A) + (not CIN);
        when "0001" => temp := ('0' & (A or B)) + (not CIN);
        when "0010" => temp := ('0' & (A or (not B))) + (not CIN);
        when "0011" => temp := ('0' & x"F") + (not CIN);
        when "0100" => temp := ('0' & A) + ('0' & (A and (not B))) + (not CIN);
        when "0101" => temp := ('0' & (A or B)) + ('0' & (A and (not B))) + (not CIN);
        when "0110" => temp := ('0' & A) - ('0' & B) - (not (not CIN));
        when "0111" => temp := ('0' & (A and (not B))) - 1 + (not CIN);
        when "1000" => temp := ('0' & A) + ('0' & (A and B)) + (not CIN);
        when "1001" => temp := ('0' & A) + ('0' & B) + (not CIN);
        when "1010" => temp := ('0' & (A or (not B))) + ('0' & (A and B)) + (not CIN);
        when "1011" => temp := ('0' & (A and B)) - 1 + (not CIN);
        when "1100" => temp := ('0' & A) + ('0' & A) + (not CIN);
        when "1101" => temp := ('0' & (A or B)) + ('0' & A) + (not CIN);
        when "1110" => temp := ('0' & (A or (not B))) + ('0' & A) + (not CIN);
        when "1111" => temp := ('0' & A) - 1 + (not CIN);
        when others => temp := (others => '0');
    end case;
end if;

Y    <= temp(3 downto 0);
COUT <= not temp(4);
if (temp(3 downto 0) = x"F") then
    AEB <= '1';
else
    AEB <= '0';
end if;
end process;
end Behavioral;

```

B.第二部分：时序逻辑器件

47. SR锁存器

功能说明

SR锁存器 (Set-Reset Latch) : S=1置位(Q=1), R=1复位(Q=0)。S=R=1为禁止状态。

真值表

S	R	Q(n+1)	状态
0	0	Q(n)	保持
0	1	0	复位
1	0	1	置位
1	1	x	禁止

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SR_LATCH is
    Port (
        S : in  STD_LOGIC;
        R : in  STD_LOGIC;
        Q : out STD_LOGIC;
        QN : out STD_LOGIC
    );
end SR_LATCH;

architecture Dataflow of SR_LATCH is
    signal q_int : STD_LOGIC;
begin
    q_int <= '0' when (S = '0' and R = '1') else
              '1' when (S = '1' and R = '0') else
              q_int;
    Q  <= q_int;
    QN <= not q_int;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SR_LATCH_Behavioral is
    Port (
        S : in  STD_LOGIC;
        R : in  STD_LOGIC;
        Q : out STD_LOGIC;
        QN : out STD_LOGIC
    );
end SR_LATCH_Behavioral;

architecture Behavioral of SR_LATCH_Behavioral is
    signal q_int : STD_LOGIC;
begin
    process(S, R)
    begin
        if (S = '1' and R = '0') then
            q_int <= '1';
        elsif (S = '0' and R = '1') then
            q_int <= '0';
        elsif (S = '0' and R = '0') then
            q_int <= q_int;
        else
            q_int <= 'X';
        end if;
    end process;
    Q <= q_int;
    QN <= not q_int;
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SR_LATCH_Structural is
    Port (
        S : in  STD_LOGIC;
        R : in  STD_LOGIC;
        Q : out STD_LOGIC;
        QN : out STD_LOGIC
    );
end SR_LATCH_Structural;

architecture Structural of SR_LATCH_Structural is
    signal nand1_out : STD_LOGIC;
    signal nand2_out : STD_LOGIC;
begin
    nand1_out <= S nand nand2_out;
    nand2_out <= R nand nand1_out;
    Q <= nand1_out;
    QN <= nand2_out;
end Structural;
```

48. D锁存器

功能说明

D锁存器：当使能E=1时透明传输($D \rightarrow Q$)，E=0时锁存当前值。

真值表

E	D	Q(n+1)
0	x	Q(n)
1	0	0
1	1	1

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_LATCH is
    Port (
        D : in  STD_LOGIC;
        E : in  STD_LOGIC;
        Q : out STD_LOGIC;
        QN : out STD_LOGIC
    );
end D_LATCH;

architecture Dataflow of D_LATCH is
    signal q_int : STD_LOGIC;
begin
    q_int <= D when E = '1' else q_int;
    Q    <= q_int;
    QN   <= not q_int;
end Dataflow;
```


行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_LATCH_Behavioral is
    Port (
        D  : in  STD_LOGIC;
        E  : in  STD_LOGIC;
        Q  : out STD_LOGIC;
        QN : out STD_LOGIC
    );
end D_LATCH_Behavioral;

architecture Behavioral of D_LATCH_Behavioral is
    signal q_int : STD_LOGIC;
begin
    process(E, D)
    begin
        if (E = '1') then
            q_int <= D;
        end if;
    end process;
    Q  <= q_int;
    QN <= not q_int;
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_LATCH_Structural is
    Port (
        D : in  STD_LOGIC;
        E : in  STD_LOGIC;
        Q : out STD_LOGIC;
        QN : out STD_LOGIC
    );
end D_LATCH_Structural;

architecture Structural of D_LATCH_Structural is
    signal s_int : STD_LOGIC;
    signal r_int : STD_LOGIC;
    signal q_int : STD_LOGIC;
begin
    s_int <= D nand E;
    r_int <= (not D) nand E;
    q_int <= s_int nand (not r_int nand q_int);
    Q <= q_int;
    QN <= not q_int;
end Structural;
```

49. D触发器

功能说明

D触发器（上升沿触发）：在时钟上升沿将D的值存入Q。

真值表

CLK	RST	D	Q(n+1)
x	1	x	0
↑	0	0	0
↑	0	1	1
0/1	0	x	Q(n)

数据流描述完整代码（异步复位）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FLIPFLOP is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        D   : in  STD_LOGIC;
        Q    : out STD_LOGIC;
        QN   : out STD_LOGIC
    );
end D_FLIPFLOP;

architecture Dataflow of D_FLIPFLOP is
    signal q_int : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            q_int <= '0';
        elsif rising_edge(CLK) then
            q_int <= D;
        end if;
    end process;
    Q <= q_int;
    QN <= not q_int;
end Dataflow;
```

行为描述完整代码（同步复位 + 时钟使能）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FLIPFLOP_CE is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        CE   : in  STD_LOGIC;
        D    : in  STD_LOGIC;
        Q    : out STD_LOGIC
    );
end D_FLIPFLOP_CE;

architecture Behavioral of D_FLIPFLOP_CE is
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            if (RST = '1') then
                Q <= '0';
            elsif (CE = '1') then
                Q <= D;
            end if;
        end if;
    end process;
end Behavioral;
```

行为描述完整代码（异步置位+复位）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FLIPFLOP_PS is
    Port (
        CLK  : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        SET  : in  STD_LOGIC;
        D    : in  STD_LOGIC;
        Q    : out STD_LOGIC
    );
end D_FLIPFLOP_PS;

architecture Behavioral of D_FLIPFLOP_PS is
begin
    process(CLK, RST, SET)
    begin
        if (RST = '1') then
            Q <= '0';
        elsif (SET = '1') then
            Q <= '1';
        elsif rising_edge(CLK) then
            Q <= D;
        end if;
    end process;
end Behavioral;
```

50. JK触发器

功能说明

JK触发器：J=K=0保持，J=0,K=1复位，J=1,K=0置位，J=K=1翻转。

真值表

CLK	RST	J	K	Q(n+1)
x	1	x	x	0
↑	0	0	0	Q(n)
↑	0	0	1	0
↑	0	1	0	1
↑	0	1	1	~Q(n)

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity JK_FLIPFLOP is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        J   : in  STD_LOGIC;
        K   : in  STD_LOGIC;
        Q   : out STD_LOGIC;
        QN  : out STD_LOGIC
    );
end JK_FLIPFLOP;

architecture Dataflow of JK_FLIPFLOP is
    signal q_int : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            q_int <= '0';
        elsif rising_edge(CLK) then
            if (J = '0' and K = '0') then
                q_int <= q_int;
            elsif (J = '0' and K = '1') then
                q_int <= '0';
            elsif (J = '1' and K = '0') then
                q_int <= '1';
            elsif (J = '1' and K = '1') then
                q_int <= not q_int;
            end if;
        end if;
    end process;
    Q <= q_int;
    QN <= not q_int;
end Dataflow;
```


行为描述完整代码（case方式）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity JK_FLIPFLOP_Case is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        J   : in  STD_LOGIC;
        K   : in  STD_LOGIC;
        Q   : out STD_LOGIC
    );
end JK_FLIPFLOP_Case;

architecture Behavioral of JK_FLIPFLOP_Case is
    signal q_int : STD_LOGIC;
    signal jk : STD_LOGIC_VECTOR(1 downto 0);
begin
    jk <= J & K;

    process(CLK, RST)
    begin
        if (RST = '1') then
            q_int <= '0';
        elsif rising_edge(CLK) then
            case jk is
                when "00" => q_int <= q_int;
                when "01" => q_int <= '0';
                when "10" => q_int <= '1';
                when "11" => q_int <= not q_int;
                when others => q_int <= q_int;
            end case;
        end if;
    end process;
    Q <= q_int;
end Behavioral;
```

51. T触发器

功能说明

T触发器：T=0保持，T=1翻转。

真值表

CLK	RST	T	Q(n+1)
x	1	x	0
↑	0	0	Q(n)
↑	0	1	~Q(n)

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity T_FLIPFLOP is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        T   : in  STD_LOGIC;
        Q    : out STD_LOGIC;
        QN   : out STD_LOGIC
    );
end T_FLIPFLOP;

architecture Dataflow of T_FLIPFLOP is
    signal q_int : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            q_int <= '0';
        elsif rising_edge(CLK) then
            if (T = '1') then
                q_int <= not q_int;
            end if;
        end if;
    end process;
    Q <= q_int;
    QN <= not q_int;
end Dataflow;
```

行为描述完整代码（同步复位）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity T_FLIPFLOP_SyncRST is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        T    : in  STD_LOGIC;
        Q    : out STD_LOGIC
    );
end T_FLIPFLOP_SyncRST;

architecture Behavioral of T_FLIPFLOP_SyncRST is
    signal q_int : STD_LOGIC;
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            if (RST = '1') then
                q_int <= '0';
            elsif (T = '1') then
                q_int <= not q_int;
            end if;
        end if;
    end process;
    Q <= q_int;
end Behavioral;
```

52. RS触发器

功能说明

RS触发器（同步）：在时钟上升沿根据R和S更新输出。

真值表

CLK	RST	S	R	Q(n+1)
x	1	x	x	0
↑	0	0	0	Q(n)
↑	0	0	1	0
↑	0	1	0	1
↑	0	1	1	禁止

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RS_FLIPFLOP is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        S   : in  STD_LOGIC;
        R   : in  STD_LOGIC;
        Q   : out STD_LOGIC;
        QN  : out STD_LOGIC
    );
end RS_FLIPFLOP;

architecture Dataflow of RS_FLIPFLOP is
    signal q_int : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            q_int <= '0';
        elsif rising_edge(CLK) then
            if (S = '1' and R = '0') then
                q_int <= '1';
            elsif (S = '0' and R = '1') then
                q_int <= '0';
            elsif (S = '0' and R = '0') then
                q_int <= q_int;
            end if;
        end if;
    end process;
    Q <= q_int;
    QN <= not q_int;
end Dataflow;
```

行为描述完整代码 (if-elsif)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RS_FLIPFLOP_Behavioral is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        S   : in  STD_LOGIC;
        R   : in  STD_LOGIC;
        Q   : out STD_LOGIC
    );
end RS_FLIPFLOP_Behavioral;

architecture Behavioral of RS_FLIPFLOP_Behavioral is
    signal q_int : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            q_int <= '0';
        elsif rising_edge(CLK) then
            if (S = '1' and R = '1') then
                q_int <= 'X';
            elsif (S = '1') then
                q_int <= '1';
            elsif (R = '1') then
                q_int <= '0';
            else
                q_int <= q_int;
            end if;
        end if;
    end process;
    Q <= q_int;
end Behavioral;
```

53. 4位寄存器

功能说明

4位并行寄存器：在时钟上升沿将4位数据存入。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity REGISTER_4BIT is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        LOAD : in  STD_LOGIC;
        D    : in  STD_LOGIC_VECTOR(3 downto 0);
        Q    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end REGISTER_4BIT;

architecture Dataflow of REGISTER_4BIT is
    signal reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                reg <= D;
            end if;
        end if;
    end process;
    Q <= reg;
end Dataflow;
```


行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity REGISTER_4BIT_Behavioral is
    Port (
        CLK  : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        LOAD : in  STD_LOGIC;
        D    : in  STD_LOGIC_VECTOR(3 downto 0);
        Q    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end REGISTER_4BIT_Behavioral;

architecture Behavioral of REGISTER_4BIT_Behavioral is
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            Q <= (others => '0');
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                Q <= D;
            end if;
        end if;
    end process;
end Behavioral;
```

结构化描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity REGISTER_4BIT_Structural is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        LOAD : in  STD_LOGIC;
        D    : in  STD_LOGIC_VECTOR(3 downto 0);
        Q    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end REGISTER_4BIT_Structural;

architecture Structural of REGISTER_4BIT_Structural is

    component D_FLIPFLOP is
        Port (CLK : in STD_LOGIC; RST : in STD_LOGIC; D : in STD_LOGIC; Q : out STD_LOGIC; QN : out STD_LOGIC);
    end component;

    signal d_mux : STD_LOGIC_VECTOR(3 downto 0);
    signal q_int : STD_LOGIC_VECTOR(3 downto 0);

begin
    d_mux(0) <= D(0) when LOAD = '1' else q_int(0);
    d_mux(1) <= D(1) when LOAD = '1' else q_int(1);
    d_mux(2) <= D(2) when LOAD = '1' else q_int(2);
    d_mux(3) <= D(3) when LOAD = '1' else q_int(3);

    FF0: D_FLIPFLOP Port Map (CLK, RST, d_mux(0), q_int(0), open);
    FF1: D_FLIPFLOP Port Map (CLK, RST, d_mux(1), q_int(1), open);
    FF2: D_FLIPFLOP Port Map (CLK, RST, d_mux(2), q_int(2), open);
    FF3: D_FLIPFLOP Port Map (CLK, RST, d_mux(3), q_int(3), open);

    Q <= q_int;
end Structural;
```

54. 8位寄存器

功能说明

8位并行寄存器：在时钟上升沿将8位数据存入。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity REGISTER_8BIT is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        LOAD : in  STD_LOGIC;
        D    : in  STD_LOGIC_VECTOR(7 downto 0);
        Q    : out STD_LOGIC_VECTOR(7 downto 0)
    );
end REGISTER_8BIT;

architecture Dataflow of REGISTER_8BIT is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                reg <= D;
            end if;
        end if;
    end process;
    Q <= reg;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity REGISTER_8BIT_Behavioral is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        LOAD : in  STD_LOGIC;
        D    : in  STD_LOGIC_VECTOR(7 downto 0);
        Q    : out STD_LOGIC_VECTOR(7 downto 0)
    );
end REGISTER_8BIT_Behavioral;

architecture Behavioral of REGISTER_8BIT_Behavioral is
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            Q <= (others => '0');
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                Q <= D;
            end if;
        end if;
    end process;
end Behavioral;
```

55. 通用寄存器

功能说明

通用寄存器支持多种操作模式：保持、并行加载、左移、右移、清零。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity UNIVERSAL_REGISTER is
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        MODE     : in  STD_LOGIC_VECTOR(2 downto 0);
        D        : in  STD_LOGIC_VECTOR(7 downto 0);
        SI_L     : in  STD_LOGIC;
        SI_R     : in  STD_LOGIC;
        Q        : out STD_LOGIC_VECTOR(7 downto 0)
    );
end UNIVERSAL_REGISTER;

architecture Dataflow of UNIVERSAL_REGISTER is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            case MODE is
                when "000" => null;
                when "001" => reg <= D;
                when "010" => reg <= reg(6 downto 0) & SI_L;
                when "011" => reg <= SI_R & reg(7 downto 1);
                when "100" => reg <= (others => '0');
                when "101" => reg <= (others => '1');
                when "110" => reg <= not reg;
                when "111" => reg <= reg(0) & reg(7 downto 1);
                when others => null;
            end case;
        end if;
    end process;
    Q <= reg;
end Dataflow;

```

56. 左移寄存器

功能说明

左移寄存器：每个时钟上升沿将数据左移一位，LSB由串行输入SI填入。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SHIFT_LEFT is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        SI  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end SHIFT_LEFT;

architecture Dataflow of SHIFT_LEFT is
    signal reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            reg <= reg(2 downto 0) & SI;
        end if;
    end process;
    Q <= reg;
end Dataflow;
```

行为描述完整代码（8位）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SHIFT_LEFT_8BIT is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        SI  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(7 downto 0)
    );
end SHIFT_LEFT_8BIT;

architecture Behavioral of SHIFT_LEFT_8BIT is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            for i in 0 to 6 loop
                reg(i) <= reg(i+1);
            end loop;
            reg(7) <= SI;
        end if;
    end process;
    Q <= reg;
end Behavioral;
```

57. 右移寄存器

功能说明

右移寄存器：每个时钟上升沿将数据右移一位，MSB由串行输入SI填入。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SHIFT_RIGHT is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        SI  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end SHIFT_RIGHT;

architecture Dataflow of SHIFT_RIGHT is
    signal reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            reg <= SI & reg(3 downto 1);
        end if;
    end process;
    Q <= reg;
end Dataflow;
```

行为描述完整代码（8位）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SHIFT_RIGHT_8BIT is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        SI  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(7 downto 0)
    );
end SHIFT_RIGHT_8BIT;

architecture Behavioral of SHIFT_RIGHT_8BIT is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            for i in 7 downto 1 loop
                reg(i) <= reg(i-1);
            end loop;
            reg(0) <= SI;
        end if;
    end process;
    Q <= reg;
end Behavioral;
```

58. 双向移位寄存器

功能说明

双向移位寄存器：根据方向控制DIR选择左移或右移。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SHIFT_BIDIR is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        DIR : in  STD_LOGIC;    -- 0=右移, 1=左移
        SI  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(7 downto 0)
    );
end SHIFT_BIDIR;

architecture Dataflow of SHIFT_BIDIR is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            if (DIR = '0') then
                reg <= SI & reg(7 downto 1);
            else
                reg <= reg(6 downto 0) & SI;
            end if;
        end if;
    end process;
    Q <= reg;
end Dataflow;
```

59. 通用移位寄存器

功能说明

通用移位寄存器：支持保持、左移、右移、并行加载四种操作模式。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SHIFT_UNIVERSAL is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        MODE : in  STD_LOGIC_VECTOR(1 downto 0);    -- 00=保持 01=右移 10=左移 11=加载
        D    : in  STD_LOGIC_VECTOR(7 downto 0);
        SI_L : in  STD_LOGIC;
        SI_R : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end SHIFT_UNIVERSAL;

architecture Dataflow of SHIFT_UNIVERSAL is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            case MODE is
                when "00" => null;
                when "01" => reg <= SI_R & reg(7 downto 1);
                when "10" => reg <= reg(6 downto 0) & SI_L;
                when "11" => reg <= D;
                when others => null;
            end case;
        end if;
    end process;
    Q <= reg;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SHIFT_UNIVERSAL_Behavioral is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        MODE : in  STD_LOGIC_VECTOR(1 downto 0);
        D    : in  STD_LOGIC_VECTOR(7 downto 0);
        SI_L : in  STD_LOGIC;
        SI_R : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end SHIFT_UNIVERSAL_Behavioral;

architecture Behavioral of SHIFT_UNIVERSAL_Behavioral is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            if (MODE = "00") then
                null;
            elsif (MODE = "01") then
                reg <= SI_R & reg(7 downto 1);
            elsif (MODE = "10") then
                reg <= reg(6 downto 0) & SI_L;
            elsif (MODE = "11") then
                reg <= D;
            end if;
        end if;
    end process;
    Q <= reg;
end Behavioral;
```

60. 串行输入并行输出

功能说明

SIPO移位寄存器：串行输入，每个时钟移入1位，4个时钟后并行输出。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SIPO is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        SI   : in  STD_LOGIC;
        Q    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end SIPO;

architecture Dataflow of SIPO is
    signal reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            reg <= SI & reg(3 downto 1);
        end if;
    end process;
    Q <= reg;
end Dataflow;
```


行为描述完整代码（8位 + 数据就绪标志）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SIPO_8BIT is
    Port (
        CLK    : in  STD_LOGIC;
        RST    : in  STD_LOGIC;
        SI     : in  STD_LOGIC;
        Q      : out STD_LOGIC_VECTOR(7 downto 0);
        READY  : out STD_LOGIC
    );
end SIPO_8BIT;

architecture Behavioral of SIPO_8BIT is
    signal reg    : STD_LOGIC_VECTOR(7 downto 0);
    signal count  : INTEGER range 0 to 7;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg    <= (others => '0');
            count  <= 0;
            READY  <= '0';
        elsif rising_edge(CLK) then
            reg <= SI & reg(7 downto 1);
            if (count = 7) then
                count <= 0;
                READY <= '1';
            else
                count <= count + 1;
                READY <= '0';
            end if;
        end if;
    end process;
    Q <= reg;
end Behavioral;
```

61. 并行输入串行输出

功能说明

PISO移位寄存器：并行加载数据，然后每个时钟串行输出1位。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PISO is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        LOAD : in  STD_LOGIC;
        D    : in  STD_LOGIC_VECTOR(3 downto 0);
        SO   : out STD_LOGIC
    );
end PISO;

architecture Dataflow of PISO is
    signal reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                reg <= D;
            else
                reg <= '0' & reg(3 downto 1);
            end if;
        end if;
    end process;
    SO <= reg(0);
end Dataflow;
```

行为描述完整代码（8位 + 状态控制）

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PISO_8BIT is
    Port (
        CLK    : in  STD_LOGIC;
        RST    : in  STD_LOGIC;
        LOAD   : in  STD_LOGIC;
        D      : in  STD_LOGIC_VECTOR(7 downto 0);
        SO     : out STD_LOGIC;
        EMPTY  : out STD_LOGIC
    );
end PISO_8BIT;

architecture Behavioral of PISO_8BIT is
    signal reg      : STD_LOGIC_VECTOR(7 downto 0);
    signal count    : INTEGER range 0 to 7;
    signal shifting : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg      <= (others => '0');
            count    <= 0;
            shifting <= '0';
            EMPTY    <= '1';
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                reg      <= D;
                count    <= 0;
                shifting <= '1';
                EMPTY    <= '0';
            elsif (shifting = '1') then
                reg <= '0' & reg(7 downto 1);
                if (count = 7) then
                    shifting <= '0';
                    EMPTY    <= '1';
                else
                    count <= count + 1;
                end if;
            end if;
        end if;
    end process;
end Behavioral;

```

```
        end if;  
    end if;  
end if;  
end process;  
S0 <= reg(0);  
end Behavioral;
```

62. 异步计数器

功能说明

4位异步（行波）计数器：每个触发器由前一级输出触发，实现 $2^4=16$ 进制计数。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ASYNC_COUNTER is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end ASYNC_COUNTER;

architecture Dataflow of ASYNC_COUNTER is
    signal q_int : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            q_int(0) <= '0';
        elsif falling_edge(CLK) then
            q_int(0) <= not q_int(0);
        end if;
    end process;

    process(q_int(0), RST)
    begin
        if (RST = '1') then
            q_int(1) <= '0';
        elsif falling_edge(q_int(0)) then
            q_int(1) <= not q_int(1);
        end if;
    end process;

    process(q_int(1), RST)
    begin
        if (RST = '1') then
            q_int(2) <= '0';
        elsif falling_edge(q_int(1)) then
            q_int(2) <= not q_int(2);
        end if;
    end process;
end

```



```
end process;

process(q_int(2), RST)
begin
    if (RST = '1') then
        q_int(3) <= '0';
    elsif falling_edge(q_int(2)) then
        q_int(3) <= not q_int(3);
    end if;
end process;

Q <= q_int;
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ASYNC_COUNTER_Behavioral is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end ASYNC_COUNTER_Behavioral;

architecture Behavioral of ASYNC_COUNTER_Behavioral is
    signal count : unsigned(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif falling_edge(CLK) then
            count <= count + 1;
        end if;
    end process;
    Q <= std_logic_vector(count);
end Behavioral;
```

63. 同步计数器

功能说明

4位同步二进制加法计数器：所有触发器共用一个时钟，带使能和进位输出。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity SYNC_COUNTER is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        EN  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0);
        TC  : out STD_LOGIC
    );
end SYNC_COUNTER;

architecture Dataflow of SYNC_COUNTER is
    signal count : unsigned(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif rising_edge(CLK) then
            if (EN = '1') then
                count <= count + 1;
            end if;
        end if;
    end process;
    Q <= std_logic_vector(count);
    TC <= '1' when (count = 15 and EN = '1') else '0';
end Dataflow;
```

行为描述完整代码（8位同步计数器）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity SYNC_COUNTER_8BIT is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        EN  : in  STD_LOGIC;
        LOAD : in  STD_LOGIC;
        D    : in  STD_LOGIC_VECTOR(7 downto 0);
        Q    : out STD_LOGIC_VECTOR(7 downto 0);
        TC   : out STD_LOGIC
    );
end SYNC_COUNTER_8BIT;

architecture Behavioral of SYNC_COUNTER_8BIT is
    signal count : unsigned(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                count <= unsigned(D);
            elsif (EN = '1') then
                count <= count + 1;
            end if;
        end if;
    end process;
    Q <= std_logic_vector(count);
    TC <= '1' when (count = 255 and EN = '1') else '0';
end Behavioral;
```

64. 十进制计数器

功能说明

BCD十进制计数器：0~9循环计数，计到9后回0并产生进位。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity DECADE_COUNTER is
    Port (
        CLK : in  STD_LOGIC;
        RST  : in  STD_LOGIC;
        EN   : in  STD_LOGIC;
        Q    : out STD_LOGIC_VECTOR(3 downto 0);
        TC   : out STD_LOGIC
    );
end DECADE_COUNTER;

architecture Dataflow of DECADE_COUNTER is
    signal count : unsigned(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif rising_edge(CLK) then
            if (EN = '1') then
                if (count = 9) then
                    count <= (others => '0');
                else
                    count <= count + 1;
                end if;
            end if;
        end if;
    end process;
    Q <= std_logic_vector(count);
    TC <= '1' when (count = 9 and EN = '1') else '0';
end Dataflow;
```

行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DECADE_COUNTER_Behavioral is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        EN  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0);
        TC  : out STD_LOGIC
    );
end DECADE_COUNTER_Behavioral;

architecture Behavioral of DECADE_COUNTER_Behavioral is
begin
    process(CLK, RST)
        variable count : INTEGER range 0 to 9;
    begin
        if (RST = '1') then
            count := 0;
            TC <= '0';
        elsif rising_edge(CLK) then
            if (EN = '1') then
                if (count = 9) then
                    count := 0;
                    TC <= '1';
                else
                    count := count + 1;
                    TC <= '0';
                end if;
            end if;
        end if;
        Q <= std_logic_vector(to_unsigned(count, 4));
    end process;
end Behavioral;
```

65. 任意进制计数器

功能说明

模N计数器 (Mod-N Counter) : 计数范围0~(N-1), 到N-1后自动归零。

数据流描述完整代码（参数化模N=6）

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity MOD_N_COUNTER is
    Generic (
        N : INTEGER := 6
    );
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        EN  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0);
        TC  : out STD_LOGIC
    );
end MOD_N_COUNTER;

architecture Dataflow of MOD_N_COUNTER is
    signal count : unsigned(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif rising_edge(CLK) then
            if (EN = '1') then
                if (count = N - 1) then
                    count <= (others => '0');
                else
                    count <= count + 1;
                end if;
            end if;
        end if;
    end process;
    Q <= std_logic_vector(count);
    TC <= '1' when (count = N - 1 and EN = '1') else '0';
end Dataflow;

```

行为描述完整代码（任意模值，Comparator实现）

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity MOD_N_COUNTER_Behavioral is
    Generic (
        MOD_VALUE : INTEGER := 10;
        WIDTH      : INTEGER := 4
    );
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        EN  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(WIDTH-1 downto 0);
        TC  : out STD_LOGIC
    );
end MOD_N_COUNTER_Behavioral;

architecture Behavioral of MOD_N_COUNTER_Behavioral is
    signal count : unsigned(WIDTH-1 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
            TC    <= '0';
        elsif rising_edge(CLK) then
            if (EN = '1') then
                if (count = MOD_VALUE - 1) then
                    count <= (others => '0');
                    TC    <= '1';
                else
                    count <= count + 1;
                    TC    <= '0';
                end if;
            else
                TC <= '0';
            end if;
        end if;
    end process;
end Behavioral;

```

```
end process;  
Q <= std_logic_vector(count);  
end Behavioral;
```

66. 可逆计数器

功能说明

可逆计数器（Up/Down Counter）：UPDN=0加计数，UPDN=1减计数。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity UPDOWN_COUNTER is
    Port (
        CLK    : in  STD_LOGIC;
        RST    : in  STD_LOGIC;
        EN     : in  STD_LOGIC;
        UPDN   : in  STD_LOGIC;      -- 0=Up, 1=Down
        Q      : out STD_LOGIC_VECTOR(3 downto 0)
    );
end UPDOWN_COUNTER;

architecture Dataflow of UPDOWN_COUNTER is
    signal count : unsigned(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif rising_edge(CLK) then
            if (EN = '1') then
                if (UPDN = '0') then
                    count <= count + 1;
                else
                    count <= count - 1;
                end if;
            end if;
        end if;
    end process;
    Q <= std_logic_vector(count);
end Dataflow;
```

行为描述完整代码（8位 + 预置数）

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity UPDOWN_COUNTER_8BIT is
    Port (
        CLK    : in  STD_LOGIC;
        RST    : in  STD_LOGIC;
        EN     : in  STD_LOGIC;
        UPDN   : in  STD_LOGIC;
        LOAD   : in  STD_LOGIC;
        D      : in  STD_LOGIC_VECTOR(7 downto 0);
        Q      : out STD_LOGIC_VECTOR(7 downto 0);
        ZERO   : out STD_LOGIC;
        MAX    : out STD_LOGIC
    );
end UPDOWN_COUNTER_8BIT;

architecture Behavioral of UPDOWN_COUNTER_8BIT is
    signal count : unsigned(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif rising_edge(CLK) then
            if (LOAD = '1') then
                count <= unsigned(D);
            elsif (EN = '1') then
                if (UPDN = '0') then
                    count <= count + 1;
                else
                    count <= count - 1;
                end if;
            end if;
        end if;
    end process;

    Q    <= std_logic_vector(count);

```



```
ZERO <= '1' when (count = 0) else '0';
MAX  <= '1' when (count = 255) else '0';
end Behavioral;
```

67. 环形计数器

功能说明

4位环形计数器：初始状态"1000"，每个时钟循环右移一位。

真值表

CLK	Q3	Q2	Q1	Q0
RST	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RING_COUNTER is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end RING_COUNTER;

architecture Dataflow of RING_COUNTER is
    signal reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= "1000";
        elsif rising_edge(CLK) then
            reg <= reg(0) & reg(3 downto 1);
        end if;
    end process;
    Q <= reg;
end Dataflow;
```

行为描述完整代码（8位 + 自校正）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RING_COUNTER_8BIT is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        EN  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(7 downto 0)
    );
end RING_COUNTER_8BIT;

architecture Behavioral of RING_COUNTER_8BIT is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= "10000000";
        elsif rising_edge(CLK) then
            if (EN = '1') then
                -- 自校正：若多个'1'则不合法，复位
                if ((reg(7) or reg(6) or reg(5) or reg(4) or
                    reg(3) or reg(2) or reg(1) or reg(0)) = '0') then
                    reg <= "10000000";
                else
                    reg <= reg(0) & reg(7 downto 1);
                end if;
            end if;
        end if;
    end process;
    Q <= reg;
end Behavioral;
```

68. Johnson计数器

功能说明

4位Johnson（扭环）计数器：将最后一级的反相输出反馈到第一级输入， $2N$ 个状态。

真值表（4位，8状态）

CLK	Q3	Q2	Q1	Q0
RST	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1
8	0	0	0	0

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity JOHNSON_COUNTER is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end JOHNSON_COUNTER;

architecture Dataflow of JOHNSON_COUNTER is
    signal reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            reg <= (not reg(0)) & reg(3 downto 1);
        end if;
    end process;
    Q <= reg;
end Dataflow;
```

行为描述完整代码（8位 Johnson）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity JOHNSON_COUNTER_8BIT is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        EN  : in  STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(7 downto 0)
    );
end JOHNSON_COUNTER_8BIT;

architecture Behavioral of JOHNSON_COUNTER_8BIT is
    signal reg : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            reg <= (others => '0');
        elsif rising_edge(CLK) then
            if (EN = '1') then
                reg <= (not reg(0)) & reg(7 downto 1);
            end if;
        end if;
    end process;
    Q <= reg;
end Behavioral;
```

69. 二分频器

功能说明

二分频器：输出频率为输入时钟频率的一半。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DIVIDER_BY2 is
    Port (
        CLK    : in  STD_LOGIC;
        RST    : in  STD_LOGIC;
        CLK_OUT : out STD_LOGIC
    );
end DIVIDER_BY2;

architecture Dataflow of DIVIDER_BY2 is
    signal toggle : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            toggle <= '0';
        elsif rising_edge(CLK) then
            toggle <= not toggle;
        end if;
    end process;
    CLK_OUT <= toggle;
end Dataflow;
```

70. 四分频器

功能说明

四分频器：输出频率为输入时钟频率的1/4。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity DIVIDER_BY4 is
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        CLK_OUT   : out STD_LOGIC
    );
end DIVIDER_BY4;

architecture Dataflow of DIVIDER_BY4 is
    signal count : unsigned(1 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
        elsif rising_edge(CLK) then
            count <= count + 1;
        end if;
    end process;
    CLK_OUT <= count(1);
end Dataflow;
```


行为描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DIVIDER_BY4_Behavioral is
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        CLK_OUT  : out STD_LOGIC
    );
end DIVIDER_BY4_Behavioral;

architecture Behavioral of DIVIDER_BY4_Behavioral is
    signal div2 : STD_LOGIC;
    signal div4 : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            div2 <= '0';
        elsif rising_edge(CLK) then
            div2 <= not div2;
        end if;
    end process;

    process(div2, RST)
    begin
        if (RST = '1') then
            div4 <= '0';
        elsif rising_edge(div2) then
            div4 <= not div4;
        end if;
    end process;

    CLK_OUT <= div4;
end Behavioral;
```

71. 任意分频器

功能说明

任意分频器：输出频率 = 输入频率 / (2*DIV)，50%占空比。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity DIVIDER_ARBITRARY is
    Generic (
        DIV : INTEGER := 5
    );
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        CLK_OUT   : out STD_LOGIC
    );
end DIVIDER_ARBITRARY;

architecture Dataflow of DIVIDER_ARBITRARY is
    signal count : unsigned(7 downto 0);
    signal clk_toggle : STD_LOGIC;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            count      <= (others => '0');
            clk_toggle <= '0';
        elsif rising_edge(CLK) then
            if (count = DIV - 1) then
                count      <= (others => '0');
                clk_toggle <= not clk_toggle;
            else
                count <= count + 1;
            end if;
        end if;
    end process;
    CLK_OUT <= clk_toggle;
end Dataflow;
```

行为描述完整代码（奇偶分频，50%占空比）

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity DIVIDER_50PERCENT is
    Generic (
        DIV_FACTOR : INTEGER := 7
    );
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        CLK_OUT  : out STD_LOGIC
    );
end DIVIDER_50PERCENT;

architecture Behavioral of DIVIDER_50PERCENT is
    signal count      : unsigned(7 downto 0);
    signal clk_pos    : STD_LOGIC;
    signal clk_neg    : STD_LOGIC;
begin

    -- 上升沿计数
    process(CLK, RST)
    begin
        if (RST = '1') then
            count <= (others => '0');
            clk_pos <= '0';
        elsif rising_edge(CLK) then
            if (count = DIV_FACTOR - 1) then
                count <= (others => '0');
            else
                count <= count + 1;
            end if;
            if (count < (DIV_FACTOR / 2)) then
                clk_pos <= '1';
            else
                clk_pos <= '0';
            end if;
        end if;
    end process;
end;

```

```
end process;

-- 下降沿计数（奇数分频用）
process(CLK, RST)
begin
    if (RST = '1') then
        clk_neg <= '0';
    elsif falling_edge(CLK) then
        if (count < (DIV_FACTOR / 2)) then
            clk_neg <= '1';
        else
            clk_neg <= '0';
        end if;
    end if;
end process;

CLK_OUT <= clk_pos or clk_neg when (DIV_FACTOR mod 2 = 1) else clk_pos;

end Behavioral;
```

72. Moore状态机

功能说明

Moore状态机：输出仅取决于当前状态。以101序列检测器为例说明。

状态转移图

S0(初始) → S1(收到1) → S2(收到10) → S3(收到101,输出1)

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MOORE_FSM is
    Port (
        CLK    : in  STD_LOGIC;
        RST    : in  STD_LOGIC;
        X      : in  STD_LOGIC;
        Z      : out STD_LOGIC
    );
end MOORE_FSM;

architecture Dataflow of MOORE_FSM is
    type state_type is (S0, S1, S2, S3);
    signal current_state : state_type;
    signal next_state    : state_type;
begin

    -- 状态寄存器
    process(CLK, RST)
    begin
        if (RST = '1') then
            current_state <= S0;
        elsif rising_edge(CLK) then
            current_state <= next_state;
        end if;
    end process;

    -- 次态逻辑
    process(current_state, X)
    begin
        case current_state is
            when S0 =>
                if (X = '1') then
                    next_state <= S1;
                else
                    next_state <= S0;
                end if;
            when S1 =>

```



```

        if (X = '0') then
            next_state <= S2;
        else
            next_state <= S1;
        end if;
    when S2 =>
        if (X = '1') then
            next_state <= S3;
        else
            next_state <= S0;
        end if;
    when S3 =>
        if (X = '1') then
            next_state <= S1;
        else
            next_state <= S2;
        end if;
    when others =>
        next_state <= S0;
    end case;
end process;

```

-- 输出逻辑（仅依赖于当前状态）

```

process(current_state)
begin
    case current_state is
        when S3 => Z <= '1';
        when others => Z <= '0';
    end case;
end process;

```

```

end Dataflow;

```

行为描述完整代码（单process实现）

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MOORE_FSM_SingleProcess is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        X   : in  STD_LOGIC;
        Z    : out STD_LOGIC
    );
end MOORE_FSM_SingleProcess;

architecture Behavioral of MOORE_FSM_SingleProcess is
    type state_type is (S0, S1, S2, S3);
    signal state : state_type;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            state <= S0;
            Z      <= '0';
        elsif rising_edge(CLK) then
            case state is
                when S0 =>
                    Z <= '0';
                    if (X = '1') then state <= S1; else state <= S0; end if;
                when S1 =>
                    Z <= '0';
                    if (X = '0') then state <= S2; else state <= S1; end if;
                when S2 =>
                    Z <= '0';
                    if (X = '1') then state <= S3; else state <= S0; end if;
                when S3 =>
                    Z <= '1';
                    if (X = '1') then state <= S1; else state <= S2; end if;
                when others =>
                    state <= S0;
                    Z      <= '0';
            end case;
        end if;
    end process;
end Behavioral;

```

```
        end if;  
    end process;  
end Behavioral;
```

73. Mealy状态机

功能说明

Mealy状态机：输出取决于当前状态和输入。以重叠101检测为例。

状态转移图

S0(初始) → S1(收到1) → S2(收到10) → 收到1则输出1并回S1

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MEALY_FSM is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        X   : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end MEALY_FSM;

architecture Dataflow of MEALY_FSM is
    type state_type is (S0, S1, S2);
    signal current_state : state_type;
    signal next_state    : state_type;
begin

    -- 状态寄存器
    process(CLK, RST)
    begin
        if (RST = '1') then
            current_state <= S0;
        elsif rising_edge(CLK) then
            current_state <= next_state;
        end if;
    end process;

    -- 次态逻辑 + 输出逻辑
    process(current_state, X)
    begin
        Z <= '0';
        case current_state is
            when S0 =>
                if (X = '1') then
                    next_state <= S1;
                else
                    next_state <= S0;
                end if;
            -- S1 and S2 logic would follow here
        end case;
    end process;
end architecture;

```

```
when S1 =>
    if (X = '0') then
        next_state <= S2;
    else
        next_state <= S1;
    end if;

when S2 =>
    if (X = '1') then
        next_state <= S1;
        Z <= '1';
    else
        next_state <= S0;
    end if;

when others =>
    next_state <= S0;
end case;
end process;
```

```
end Dataflow;
```

行为描述完整代码（一process式）


```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MEALY_FSM_Behavioral is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        X   : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end MEALY_FSM_Behavioral;

architecture Behavioral of MEALY_FSM_Behavioral is
    type state_type is (S0, S1, S2);
    signal state : state_type;
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            state <= S0;
            Z     <= '0';
        elsif rising_edge(CLK) then
            case state is
                when S0 =>
                    Z <= '0';
                    if (X = '1') then state <= S1; end if;
                when S1 =>
                    Z <= '0';
                    if (X = '0') then state <= S2; end if;
                when S2 =>
                    if (X = '1') then
                        Z <= '1';
                        state <= S1;
                    else
                        Z <= '0';
                        state <= S0;
                    end if;
                when others =>
                    state <= S0; Z <= '0';
            end case;
        end if;
    end process;
end Behavioral;

```

```
        end case;  
    end if;  
end process;  
end Behavioral;
```

74. 序列检测器1011

功能说明

Moore型重叠序列检测器：检测串行输入中的"1011"模式，检测到时输出1。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SEQ_DETECTOR_1011 is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        X   : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end SEQ_DETECTOR_1011;

architecture Dataflow of SEQ_DETECTOR_1011 is
    type state_type is (S0, S1, S10, S101, S1011);
    signal current_state : state_type;
    signal next_state    : state_type;
begin

    process(CLK, RST)
    begin
        if (RST = '1') then
            current_state <= S0;
        elsif rising_edge(CLK) then
            current_state <= next_state;
        end if;
    end process;

    process(current_state, X)
    begin
        Z <= '0';
        case current_state is
            when S0 =>
                if (X = '1') then next_state <= S1;
                else next_state <= S0; end if;

            when S1 =>
                if (X = '0') then next_state <= S10;
                else next_state <= S1; end if;
        end case;
    end process;
end architecture;

```

```
when S10 =>
    if (X = '1') then next_state <= S101;
    else next_state <= S0; end if;

when S101 =>
    if (X = '1') then
        next_state <= S1011;
        Z <= '1';
    else
        next_state <= S10;
    end if;

when S1011 =>
    if (X = '1') then next_state <= S1;
    else next_state <= S10; end if;

when others =>
    next_state <= S0;
end case;
end process;
```

```
end Dataflow;
```

行为描述完整代码（移位寄存器实现）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SEQ_DETECTOR_1011_SR is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        X   : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end SEQ_DETECTOR_1011_SR;

architecture Behavioral of SEQ_DETECTOR_1011_SR is
    signal shift_reg : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK, RST)
    begin
        if (RST = '1') then
            shift_reg <= (others => '0');
        elsif rising_edge(CLK) then
            shift_reg <= X & shift_reg(3 downto 1);
        end if;
    end process;

    Z <= '1' when (shift_reg = "1011") else '0';
end Behavioral;
```

75. 序列检测器1101

功能说明

重叠序列检测器：检测串行输入中的"1101"模式。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SEQ_DETECTOR_1101 is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        X   : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end SEQ_DETECTOR_1101;

architecture Dataflow of SEQ_DETECTOR_1101 is
    type state_type is (S0, S1, S11, S110, S1101);
    signal current_state : state_type;
    signal next_state    : state_type;
begin

    process(CLK, RST)
    begin
        if (RST = '1') then
            current_state <= S0;
        elsif rising_edge(CLK) then
            current_state <= next_state;
        end if;
    end process;

    process(current_state, X)
    begin
        Z <= '0';
        case current_state is
            when S0 =>
                if (X = '1') then next_state <= S1;
                else next_state <= S0; end if;

            when S1 =>
                if (X = '1') then next_state <= S11;
                else next_state <= S0; end if;
        end case;
    end process;
end architecture;

```



```
when S11 =>
    if (X = '0') then next_state <= S110;
    else next_state <= S11; end if;

when S110 =>
    if (X = '1') then
        next_state <= S1101;
        Z <= '1';
    else
        next_state <= S0;
    end if;

when S1101 =>
    if (X = '1') then next_state <= S11;
    else next_state <= S0; end if;

when others =>
    next_state <= S0;
end case;
end process;
```

```
end Dataflow;
```

行为描述完整代码（Mealy型，三process）

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SEQ_DETECTOR_1101_Mealy is
    Port (
        CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        X   : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end SEQ_DETECTOR_1101_Mealy;

architecture Behavioral of SEQ_DETECTOR_1101_Mealy is
    type state_type is (S0, S1, S11, S110);
    signal current_state : state_type;
    signal next_state    : state_type;
begin

    -- 状态寄存器
    process(CLK, RST)
    begin
        if (RST = '1') then
            current_state <= S0;
        elsif rising_edge(CLK) then
            current_state <= next_state;
        end if;
    end process;

    -- 次态逻辑
    process(current_state, X)
    begin
        case current_state is
            when S0 =>
                if (X = '1') then next_state <= S1;
                else next_state <= S0; end if;
            when S1 =>
                if (X = '1') then next_state <= S11;
                else next_state <= S0; end if;
            when S11 =>

```

```

        if (X = '1') then next_state <= S11;
        else next_state <= S110; end if;
    when S110 =>
        if (X = '1') then next_state <= S1;
        else next_state <= S0; end if;
    when others =>
        next_state <= S0;
    end case;
end process;

-- 输出逻辑
process(current_state, X)
begin
    if (current_state = S110 and X = '1') then
        Z <= '1';
    else
        Z <= '0';
    end if;
end process;

end Behavioral;

```

76. ROM

功能说明

只读存储器：256x8位ROM，存储固定数据（如正弦波查找表）。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ROM_256x8 is
    Port (
        ADDR : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA : out STD_LOGIC_VECTOR(7 downto 0)
    );
end ROM_256x8;

architecture Dataflow of ROM_256x8 is
    type rom_type is array (0 to 255) of STD_LOGIC_VECTOR(7 downto 0);

    constant ROM_CONTENT : rom_type := (
        0   => x"00",
        1   => x"01",
        2   => x"02",
        3   => x"03",
        4   => x"04",
        5   => x"05",
        6   => x"06",
        7   => x"07",
        8   => x"08",
        9   => x"09",
        10  => x"0A",
        11  => x"0B",
        12  => x"0C",
        13  => x"0D",
        14  => x"0E",
        15  => x"0F",
        16  => x"10",
        17  => x"11",
        18  => x"12",
        19  => x"13",
        20  => x"14",
        21  => x"15",
        22  => x"16",
        23  => x"17",

```

```
24 => x"18",  
25 => x"19",  
26 => x"1A",  
27 => x"1B",  
28 => x"1C",  
29 => x"1D",  
30 => x"1E",  
31 => x"1F",  
32 => x"20",  
others => x"00"  
);
```

```
begin  
    DATA <= ROM_CONTENT(to_integer(unsigned(ADDR)));  
end Dataflow;
```

行为描述完整代码（参数化ROM）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ROM_PARAM is
    Generic (
        ADDR_WIDTH : INTEGER := 8;
        DATA_WIDTH : INTEGER := 8
    );
    Port (
        ADDR : in  STD_LOGIC_VECTOR(ADDR_WIDTH-1 downto 0);
        DATA : out STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0)
    );
end ROM_PARAM;

architecture Behavioral of ROM_PARAM is
    constant DEPTH : INTEGER := 2**ADDR_WIDTH;
    type rom_type is array (0 to DEPTH-1) of STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);

    function init_rom return rom_type is
        variable mem : rom_type;
    begin
        for i in 0 to DEPTH-1 loop
            mem(i) := std_logic_vector(to_unsigned(i mod 256, DATA_WIDTH));
        end loop;
        return mem;
    end function;

    constant ROM_DATA : rom_type := init_rom;

begin
    DATA <= ROM_DATA(to_integer(unsigned(ADDR)));
end Behavioral;
```

77. 单端口RAM

功能说明

单端口RAM：同一时刻只能读或写，256x8位。

数据流描述完整代码

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RAM_SP_256x8 is
    Port (
        CLK      : in  STD_LOGIC;
        WE       : in  STD_LOGIC;
        ADDR     : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_IN  : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_OUT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end RAM_SP_256x8;

architecture Dataflow of RAM_SP_256x8 is
    type ram_type is array (0 to 255) of STD_LOGIC_VECTOR(7 downto 0);
    signal ram : ram_type := (others => (others => '0'));
    signal addr_reg : unsigned(7 downto 0);
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            if (WE = '1') then
                ram(to_integer(unsigned(ADDR))) <= DATA_IN;
            end if;
            addr_reg <= unsigned(ADDR);
        end if;
    end process;
    DATA_OUT <= ram(to_integer(addr_reg));
end Dataflow;
```

行为描述完整代码（Read-First模式）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RAM_SP_READFIRST is
    Port (
        CLK      : in  STD_LOGIC;
        WE       : in  STD_LOGIC;
        ADDR     : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_IN  : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_OUT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end RAM_SP_READFIRST;

architecture Behavioral of RAM_SP_READFIRST is
    type ram_type is array (0 to 255) of STD_LOGIC_VECTOR(7 downto 0);
    signal ram : ram_type := (others => (others => '0'));
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            DATA_OUT <= ram(to_integer(unsigned(ADDR)));
            if (WE = '1') then
                ram(to_integer(unsigned(ADDR))) <= DATA_IN;
            end if;
        end if;
    end process;
end Behavioral;
```

行为描述完整代码（Write-First模式）

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RAM_SP_WRITEFIRST is
    Port (
        CLK      : in  STD_LOGIC;
        WE       : in  STD_LOGIC;
        ADDR     : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_IN  : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_OUT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end RAM_SP_WRITEFIRST;

architecture Behavioral of RAM_SP_WRITEFIRST is
    type ram_type is array (0 to 255) of STD_LOGIC_VECTOR(7 downto 0);
    signal ram : ram_type := (others => (others => '0'));
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            if (WE = '1') then
                ram(to_integer(unsigned(ADDR))) <= DATA_IN;
                DATA_OUT <= DATA_IN;
            else
                DATA_OUT <= ram(to_integer(unsigned(ADDR)));
            end if;
        end if;
    end process;
end Behavioral;
```

78. 双端口RAM

功能说明

双端口RAM：两个独立的读/写端口，可同时从不同地址读写。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RAM_DP is
    Port (
        CLK      : in  STD_LOGIC;
        WE_A     : in  STD_LOGIC;
        ADDR_A    : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_IN_A : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_OUT_A : out STD_LOGIC_VECTOR(7 downto 0);

        WE_B     : in  STD_LOGIC;
        ADDR_B    : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_IN_B : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_OUT_B : out STD_LOGIC_VECTOR(7 downto 0)
    );
end RAM_DP;

architecture Dataflow of RAM_DP is
    type ram_type is array (0 to 255) of STD_LOGIC_VECTOR(7 downto 0);
    shared variable ram : ram_type := (others => (others => '0'));
begin

    -- 端口A
    process(CLK)
    begin
        if rising_edge(CLK) then
            if (WE_A = '1') then
                ram(to_integer(unsigned(ADDR_A))) := DATA_IN_A;
            end if;
            DATA_OUT_A <= ram(to_integer(unsigned(ADDR_A)));
        end if;
    end process;

    -- 端口B
    process(CLK)
    begin
        if rising_edge(CLK) then

```

```
    if (WE_B = '1') then
        ram(to_integer(unsigned(ADDR_B))) := DATA_IN_B;
    end if;
    DATA_OUT_B <= ram(to_integer(unsigned(ADDR_B)));
end if;
end process;

end Dataflow;
```


行为描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RAM_DP_Behavioral is
    Port (
        CLK      : in  STD_LOGIC;
        WE_A      : in  STD_LOGIC;
        ADDR_A    : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_IN_A : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_OUT_A : out STD_LOGIC_VECTOR(7 downto 0);

        WE_B      : in  STD_LOGIC;
        ADDR_B    : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_IN_B : in  STD_LOGIC_VECTOR(7 downto 0);
        DATA_OUT_B : out STD_LOGIC_VECTOR(7 downto 0)
    );
end RAM_DP_Behavioral;

architecture Behavioral of RAM_DP_Behavioral is
    type ram_type is array (0 to 255) of STD_LOGIC_VECTOR(7 downto 0);
    signal ram : ram_type := (others => (others => '0'));
begin

    process(CLK)
    begin
        if rising_edge(CLK) then
            if (WE_A = '1') then
                ram(to_integer(unsigned(ADDR_A))) <= DATA_IN_A;
            end if;
            DATA_OUT_A <= ram(to_integer(unsigned(ADDR_A)));
        end if;
    end process;

    process(CLK)
    begin
        if rising_edge(CLK) then
            if (WE_B = '1') then
                ram(to_integer(unsigned(ADDR_B))) <= DATA_IN_B;
            end if;
        end if;
    end process;
end architecture Behavioral;

```

```
        end if;  
        DATA_OUT_B <= ram(to_integer(unsigned(ADDR_B)));  
    end if;  
end process;  
  
end Behavioral;
```

79. FIFO

功能说明

同步FIFO（First-In-First-Out）：先进先出队列，16x8位深度。

数据流描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity FIFO_SYNC is
    Generic (
        DATA_WIDTH : INTEGER := 8;
        FIFO_DEPTH  : INTEGER := 16
    );
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        WR_EN    : in  STD_LOGIC;
        RD_EN    : in  STD_LOGIC;
        DATA_IN : in  STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);
        DATA_OUT : out STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);
        FULL     : out STD_LOGIC;
        EMPTY    : out STD_LOGIC
    );
end FIFO_SYNC;

architecture Dataflow of FIFO_SYNC is
    type fifo_mem_type is array (0 to FIFO_DEPTH-1) of STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);
    signal fifo_mem : fifo_mem_type := (others => (others => '0'));
    signal wr_ptr   : unsigned(3 downto 0);
    signal rd_ptr   : unsigned(3 downto 0);
    signal fifo_cnt : unsigned(4 downto 0);
begin

    process(CLK, RST)
    begin
        if (RST = '1') then
            wr_ptr   <= (others => '0');
            rd_ptr   <= (others => '0');
            fifo_cnt <= (others => '0');

            elsif rising_edge(CLK) then

                -- 写操作

```

```

if (WR_EN = '1' and fifo_cnt < FIFO_DEPTH) then
    fifo_mem(to_integer(wr_ptr)) <= DATA_IN;
    wr_ptr <= wr_ptr + 1;
end if;

-- 读操作
if (RD_EN = '1' and fifo_cnt > 0) then
    rd_ptr <= rd_ptr + 1;
end if;

-- 计数器更新
if (WR_EN = '1' and RD_EN = '0' and fifo_cnt < FIFO_DEPTH) then
    fifo_cnt <= fifo_cnt + 1;
elsif (RD_EN = '1' and WR_EN = '0' and fifo_cnt > 0) then
    fifo_cnt <= fifo_cnt - 1;
end if;

end if;
end process;

DATA_OUT <= fifo_mem(to_integer(rd_ptr));

FULL  <= '1' when (fifo_cnt = FIFO_DEPTH) else '0';
EMPTY <= '1' when (fifo_cnt = 0) else '0';

end Dataflow;

```

行为描述完整代码

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity FIFO_SYNC_Behavioral is
    Generic (
        DATA_WIDTH : INTEGER := 8;
        FIFO_DEPTH : INTEGER := 16
    );
    Port (
        CLK      : in  STD_LOGIC;
        RST      : in  STD_LOGIC;
        WR_EN    : in  STD_LOGIC;
        RD_EN    : in  STD_LOGIC;
        DATA_IN : in  STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);
        DATA_OUT : out STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);
        FULL     : out STD_LOGIC;
        EMPTY    : out STD_LOGIC;
        ALMOST_FULL : out STD_LOGIC;
        ALMOST_EMPTY : out STD_LOGIC
    );
end FIFO_SYNC_Behavioral;

architecture Behavioral of FIFO_SYNC_Behavioral is
    type fifo_mem_type is array (0 to FIFO_DEPTH-1) of STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);
    signal fifo_mem : fifo_mem_type := (others => (others => '0'));
    signal wr_ptr   : INTEGER range 0 to FIFO_DEPTH-1;
    signal rd_ptr   : INTEGER range 0 to FIFO_DEPTH-1;
    signal fifo_cnt : INTEGER range 0 to FIFO_DEPTH;
begin

    process(CLK, RST)
    begin
        if (RST = '1') then
            wr_ptr   <= 0;
            rd_ptr   <= 0;
            fifo_cnt <= 0;

            elsif rising_edge(CLK) then

```



```

    if (WR_EN = '1' and fifo_cnt < FIFO_DEPTH) then
        fifo_mem(wr_ptr) <= DATA_IN;
        if (wr_ptr = FIFO_DEPTH - 1) then
            wr_ptr <= 0;
        else
            wr_ptr <= wr_ptr + 1;
        end if;
    end if;

    if (RD_EN = '1' and fifo_cnt > 0) then
        if (rd_ptr = FIFO_DEPTH - 1) then
            rd_ptr <= 0;
        else
            rd_ptr <= rd_ptr + 1;
        end if;
    end if;

    if (WR_EN = '1' and RD_EN = '0' and fifo_cnt < FIFO_DEPTH) then
        fifo_cnt <= fifo_cnt + 1;
    elsif (RD_EN = '1' and WR_EN = '0' and fifo_cnt > 0) then
        fifo_cnt <= fifo_cnt - 1;
    end if;

end if;
end process;

DATA_OUT    <= fifo_mem(rd_ptr);
FULL        <= '1' when (fifo_cnt = FIFO_DEPTH) else '0';
EMPTY       <= '1' when (fifo_cnt = 0) else '0';
ALMOST_FULL <= '1' when (fifo_cnt >= FIFO_DEPTH - 2) else '0';
ALMOST_EMPTY <= '1' when (fifo_cnt <= 2) else '0';

end Behavioral;

```